shieldify



SMART CONTRACT SECURITY ASSESSMENT

Rini

SECURITY REVIEW

23 SEPTEMBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1 **About Shieldify** 3

2 **Disclaimer** 3

3 **About Rini** 3

4 **Risk Classification** 4

4.1 Impact 4

4.2 Likelihood 4

5 **Security Review Summary** 4

5.1 Protocol Summary 5

5.2 Scope 5

6 **Findings Summary** 6

7 **Findings** 8

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Rini

Rini connects asynchronous ERC-7540 vaults on HyperEVM to leader-operated trading strategies on HyperCore. The protocol deliberately separates depositor access, trading authority, valuation, and settlement so that no single role holds both custody and discretion.

HyperEVM and HyperCore are separate, non-atomic state machines. A successful EVM transaction records only that an action was submitted; it never proves that the requested HyperCore effect completed. Every capital movement is therefore modelled as an explicit lifecycle – record intent, submit, observe the destination, reconcile once, then complete accounting.

At a high level, the main functionalities are:

- Depositors submit `requestDeposit()` and `requestRedeem()` intents against a Lagoon ERC-7540 vault; shares and assets become claimable only after settlement against an accepted valuation.
- A Rini-controlled Safe holds idle EVM assets and is the only party that settles the vault. It grants `CuratorGuard` an allowance for trading allocation.
- `CuratorGuard` initiates EVM-to-Core allocation, tracks pending cross-machine flows, and restricts all returns to the immutable Safe.
- A constrained API agent gives the leader trading authority over the HyperCore account without any Safe, Guard, Lagoon, or settlement role.

- An off-chain keeper reads both machines, composes a dual-source NAV, reconciles in-flight movement, proposes valuations, and triggers operations. It cannot settle outside on-chain bounds or transfer arbitrarily.
- A Guardian can reduce exposure, veto delayed risk increases, and run specified emergency actions, but can never raise exposure or acquire transfer authority.

Valuation is dual-source: HyperCore is read from two independent vantages and cross-checked against EVM state, producing one reliability grade of `FRESH`, `STALE`, or `DEGRADED`. Only a `FRESH` valuation may be proposed, and only a `FRESH` valuation permits settlement. Auto-deleveraging is detected by reconciling expected against observed position state rather than by trusting any single flag, and a suspected event moves the vault into a degraded operational state that blocks new deposit settlement.

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	CRITICAL	HIGH	MEDIUM
Likelihood: Medium	HIGH	MEDIUM	LOW
Likelihood: Low	MEDIUM	LOW	LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 14 days with a total of 336 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying five Medium and eighteen Low issues. They are mainly related to keeper-side accounting of asynchronous request lifecycles, keeper adapters bound to a superseded contract generation, role-identity reservation and expiry across the governance timelock families, and availability gaps in escalation, alerting, and liquidity preflight paths.

The Rini team has done a great job with their architecture documentation and test suite, and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	RINI
Repository	rini-audit-r1
Type of Project	Asynchronous ERC-7540 Vault & Delegated Trading Infrastructure
Security Review Timeline	14 days
Review Commit Hash	fa87871bb5a909929a2b065e1abacc083a37eafa
Fixes Review Commit Hash	a43e49a9710ec9575679a2927591c43311a988d8

5.2 Scope

The review covered the following components of the Rini stack:

COMPONENT	DESCRIPTION
`CuratorGuard`	EVM-to-Core allocation, pending-flow accounting, agent slot, and return-to-Safe policy
`VaultStackFactory`	Deterministic deployment and genesis wiring of a complete vault stack
`VaultOwnerTimelock`	Delayed, vetoable rotation of vault-owner and protocol-role identities
`LeaderAllowlist`	Management and validation of authorized protocol leaders
`RiniSafeGuard`	Safe transaction and module guard restricting the Safe's permitted call surface
`RiniSettleModule`	Safe module implementing the settlement boundary against the Lagoon vault
Supporting libraries	`VaultStackChecks`, `RolesWiring`, `Precompiles`, `CoreActions`, and `SafeGuardAllowance`

COMPONENT	DESCRIPTION
Keeper service	Indexing, NAV calculation, reconciliation, settlement driving, bridge operations, and alerting
Lagoon ERC-7540 integration	Integration surface for asynchronous requests, settlement and claims; the underlying Lagoon implementation was not in scope

The audited surface spans the on-chain Solidity contracts under `contracts/src` and the off-chain keeper service under `services/keeper/src`. Deployment and operational scripts under `contracts/script` were reviewed only where explicitly included in the engagement scope.

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 5
- **Low** issues: 18
- **Info** issues: 9

ID	TITLE	SEVERITY	STATUS
[M-01]	Same-Epoch Redeem Increments Overwrite the Keeper's Request Amount and Bypass Its Liquidity Preflight	MEDIUM	FIXED
[M-02]	The Keeper Cannot Express or Verify the Signer Topology Required by V4	MEDIUM	FIXED
[M-03]	The Real Keeper Settlement Transport Cannot Execute Against Factory-Created V4 Stacks	MEDIUM	FIXED
[M-04]	A Proposed-Agent Identity Pin Can Repeatedly Block Steward-Owned Role Rotations	MEDIUM	FIXED
[M-05]	The Real Bridge Driver Reads `CuratorGuard` Through a Pre-V4 Surface	MEDIUM	FIXED
[L-01]	Whitelist-Mode Stacks Ship With Unusable Safe Claim-on-Behalf Helpers	LOW	FIXED
[L-02]	Minting the Timelock as Whitelist Manager Blocks Steward Role Rotations	LOW	FIXED
[L-03]	Guard-Lifecycle Nonce Monitor Does Not Re-Arm After Recovery	LOW	FIXED

ID	TITLE	SEVERITY	STATUS
[L-04]	Hot-Adopted Liquidity Policies Bypass the Autonomous-Liquidity Admission Interlock	LOW	FIXED
[L-05]	A Stale Liquidity-Policy Worker Can Clear a Newer Shortfall Episode	LOW	FIXED
[L-06]	Commissioning Can Mutate a Foreign-Chain Registry Row	LOW	FIXED
[L-07]	Guardian Veto Alerts Collide Across Vaults During Fleet-Wide Rotations	LOW	FIXED
[L-08]	Fill Pagination Can Certify Incomplete ADL and Builder-Fee Coverage	LOW	FIXED
[L-09]	KMS Signing Calls Are Constructed With No Request Deadline	LOW	FIXED
[L-10]	Live Requests Can Disappear Because the Indexer Misorders Same-Epoch Re-Requests	LOW	FIXED
[L-11]	Bridge Escalations Can Be Suppressed Because Orphan Retries Abort the Tick	LOW	FIXED
[L-12]	Safe Guard Permits an Irreversible USDC Transfer to the Vault	LOW	FIXED
[L-13]	Whitelist Administration Can Be Disabled Because Factory Validation Accepts Inert Managers	LOW	FIXED
[L-14]	Expired Role Proposals Can Permanently Block Subsequent Role Rotations	LOW	FIXED
[L-15]	Unsafe Reconciliation Can Go Unwarned Because Residual Sweeps Are Not Conserved	LOW	FIXED
[L-16]	Returns Can Fail and Block the Bridge Lane Because Held Collateral Is Counted as Transferable	LOW	FIXED
[L-17]	Lifecycle State Can Become Permanently Stale Because Malformed Known Events Advance Cursors	LOW	FIXED
[L-18]	Redemptions Can Be Delayed Because Direct Silo Share Transfers Bypass Keeper Coverage	LOW	ACKNOWLEDGED
[I-01]	Two Role Changes for the Same Address Can Block Each Other	INFO	ACKNOWLEDGED

ID	TITLE	SEVERITY	STATUS
[I-02]	Fee-Receiver Rotation Omits Collision and Lagoon Admission Checks	INFO	ACKNOWLEDGED
[I-03]	No Boot Admission Check Ties Operated-Vault Count to the Read Budget the Repository Itself Models	INFO	FIXED
[I-04]	`proposeRaiseRates` Binds a Steward-Callable Operation to the Leader Epoch	INFO	FIXED
[I-05]	Agent-Rotation Activation Silently Supersedes a Pending Agent and Names Only the New One	INFO	FIXED
[I-06]	The Redemption Confirmation Path Overwrites the Recorded Transaction Hash on Replay	INFO	FIXED
[I-07]	Redemptions Can Remain Pending Because Silent No-Ops Emit Generic Success	INFO	FIXED
[I-08]	Role Changes Can Be Delayed Because the Timelock Retains a Cleared Sanctions Oracle	INFO	FIXED
[I-09]	V4 Timelock Operations Can Be Missed Because the Watcher Uses a Stale Event ABI	INFO	FIXED

7 Findings

M-01

MEDIUM FIXED

Same-Epoch Redeem Increments Overwrite the Keeper's Request Amount and Bypass Its Liquidity Preflight

Description

Lagoon allows a controller to submit multiple redeem requests during the same epoch. Each call adds its shares to the same on-chain request but emits an event containing only that call's increment.

The keeper stores one withdrawal row per `(vaultId, requestId, controller)`. When it processes another event for the same request, it replaces the row's `shares` with the latest increment instead of preserving the accumulated amount.

For example, a user can request a large redemption and then add a one-share redemption. Lagoon records the full sum, while the keeper records only one share.

Both the keeper's liquidity policy and its pre-settlement balance and allowance checks use this understated database value. Lagoon later settles against the full on-chain amount. If the Safe can cover the keeper's estimate but not the actual payout, Lagoon silently returns without settling. The keeper classifies the missing settlement event as `settle_noop` and places the vault under a durable custody hold.

Location of Affected Code

Lagoon accumulates the shares but emits only the latest increment:

```
// contracts/lib/lagoon-v0/src/v0.6.0/ERC7540.sol
$.epochs[_redeemId].redeemRequest[controller] += shares;

_update(owner, address($.pendingSilo), shares);

emit RedeemRequest(controller, owner, _redeemId, msg.sender, shares);
```

The keeper saves the individual event amount:

```
// services/keeper/src/application/confirm-pending-withdrawal.ts
const withdrawal: WithdrawalEntity = {
 ref: event.ref,
 vaultId: event.vaultId,
 owner: event.ref.controller,
 shares: event.shares,
 // ...
}
```

The database upsert replaces the previous amount:

```
// packages/db/src/repositories/pending-withdrawal.repository.ts
.onConflictDoUpdate({
 target: [
  pendingWithdrawals.vaultId,
  pendingWithdrawals.requestId,
  pendingWithdrawals.controller,
 ],
 set: {
  status: row.status,
  shares: row.shares,
  // ...
 },
})
```

The settlement preflight then trusts the overwritten amount:

```
// services/keeper/src/application/process-withdrawal.ts
const pendingShares = pending.reduce((acc, w) => acc + w.shares, 0n)
const payout = estimateRedeemPayout(pendingShares, nav, supply.right)
```

Lagoon instead checks the full batch frozen from the pending silo:

```
// contracts/lib/lagoon-v0/src/v0.6.0/libraries/ERC7540Lib.sol
uint256 pendingShares = $.settles[redeemSettleId].pendingShares;
uint256 assetsToWithdraw =
  IERC4626(address(this)).convertToAssets(pendingShares - exitFeeShares);

uint256 assetsInTheSafe = IERC20(_asset).balanceOf(assetsCustodian);
if (assetsToWithdraw == 0 || assetsToWithdraw > assetsInTheSafe) return;
```

Impact

Any shareholder can make the keeper materially understate a pending redemption. This can bypass the liquidity preflight and turn an ordinary funding shortfall into a vault-wide custody hold that blocks both deposit and redemption settlement until manual recovery.

The autonomous liquidity policy uses the same understated amount, so it may also return insufficient USDC to resolve the real obligation.

A second, independent route to the same understated-coverage outcome is described in L-18 (Redemptions Can Be Delayed Because Direct Silo Share Transfers Bypass Keeper Coverage); the two findings require separate fixes.

Proof of Concept

1. A controller submits a large redeem request for `99,999` shares.
2. Before the epoch advances, the same controller submits another redeem request for `1` share.
3. Lagoon accumulates both calls into one request for `100,000` shares.
4. The keeper indexes both events into the same database row. The second upsert replaces `99,999` with `1`.
5. Assume the Safe has enough USDC for the keeper's one-share estimate but not for the real `100,000`-share payout. Also assume its allowance is sufficient for the real payout.
6. The keeper's coverage preflight passes because it prices only one share.
7. Lagoon prices all `100,000` shares and returns without emitting `SettleRedeem` because the Safe balance is insufficient.
8. The keeper classifies the successful transaction without a settlement event as `settle_noop` and sets the durable custody hold.

If the Safe's allowance is also below the actual payout, settlement reverts instead of silently returning, but the keeper still installs the same custody hold.

Recommendation

Persist redeem increments with a replay-safe event identity such as `(vaultId, txHash, logIndex)`, and derive the request's total shares from those unique increments.

Carry `logIndex` into the normalized `RedeemRequested` event and update the aggregate request amount only when that event has not previously been processed. The event record and aggregate update should be atomic.

Add regression coverage for:

1. two different increments sharing one request ID;
2. replaying the same event without double-counting;
3. the balance-preflight undercount; and
4. the allowance-preflight undercount.

Team Response

Fixed.

M-02

MEDIUM FIXED

The Keeper Cannot Express or Verify the Signer Topology Required by V4

Description

V4 assigns both `CuratorGuard` lifecycle actions and `RiniSettleModule` settlement to the same `settleTrigger()` identity. The keeper still resolves these duties through the legacy four-key `KeeperKmsIdentity` model.

Both production resolvers call `assertDistinctIdentities(kmsClients)` without the sharing allowance the function already supports. Its docstring names the `GuardLifecycle` / `SettleTrigger` fusion as the intended exception, and no production call site passes it. A correctly fused V4 key therefore fails keeper startup at this check. Configuring distinct keys boots the keeper, but only one key can equal the on-chain `settleTrigger()`, so the other write path reverts with `Unauthorized` or `NotSettleTrigger` on every transaction.

The keeper also never reads `settleAuthority()` or `settleTrigger()` before arming these writers, so it cannot detect that its configured signer differs from the current on-chain authority. The fused topology is the canonical one: the hot-signing topology runbook states that the bridge driver is fused into `settleTrigger()` and that there is no third hot bridge identity.

Location of Affected Code

File: `services/keeper/src/infrastructure/build-deps.ts`

```
yield* assertDistinctIdentities(kmsClients)
const propose = yield* makeKmsViemAccount(kmsClients[KeeperKmsIdentity.ValuationPropose])
const settle = yield* makeKmsViemAccount(kmsClients[KeeperKmsIdentity.SettleTrigger])
```

```
yield* assertDistinctIdentities(kmsClients)
return yield* makeKmsViemAccount(kmsClients[KeeperKmsIdentity.GuardLifecycle])
```

File: `contracts/src/CuratorGuard.sol`

```
modifier onlySettleTrigger() {
  if (msg.sender != settleTrigger) revert Unauthorized();
  _;
}
```

File: `contracts/src/RiniSettleModule.sol`

```
(address expectedTrigger, uint256 expectedEpoch) = _settleAuthority();
if (msg.sender != expectedTrigger) revert NotSettleTrigger(msg.sender);
```

Impact

A factory-created V4 stack has no signer configuration that satisfies both the keeper's boot checks and the contracts' authority model. Every configuration fails one way or the other: a fused key stops the keeper at startup, and split keys leave settlement or Guard lifecycle writes reverting on-chain. The failure is fail-closed. No funds are exposed.

Proof of Concept

Save as `services/keeper/src/infrastructure/adapters/M02-v4-fused-identity.test.ts`:

```
import { Effect, Either } from 'effect'
import { describe, expect, it } from 'vitest'
import {
 type Hex,
 keccak256,
 recoverPublicKey,
 serializeSignature,
 toHex,
} from 'viem'
import { sign } from 'viem/accounts'
```

```
import {
  type KeeperKmsClients,
  KeeperKmsIdentity,
  assertDistinctIdentities,
} from './kms-signer-set.adapter'
import type { KmsClient } from './kms-signer.adapter'

const KEYS = {
  [KeeperKmsIdentity.ValuationPropose]:
    '0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d',
  [KeeperKmsIdentity.GuardLifecycle]:
    '0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a',
  [KeeperKmsIdentity.SettleTrigger]:
    '0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6',
  [KeeperKmsIdentity.WatchdogCanary]:
    '0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a',
} satisfies Record<string, Hex>

async function publicKey(privateKey: Hex): Promise<Hex> {
  const hash = keccak256(toHex('M-01 public-key probe'))
  const signature = await sign({ hash, privateKey })
  return recoverPublicKey({ hash, signature: serializeSignature(signature) })
}

function client(privateKey: Hex): KmsClient {
  return {
    getPublicKey: () => publicKey(privateKey),
    sign: async () => '0x',
  }
}

function fusedClients(): KeeperKmsClients {
  return {
    [KeeperKmsIdentity.ValuationPropose]: client(KEYS[KeeperKmsIdentity.ValuationPropose]),
    [KeeperKmsIdentity.GuardLifecycle]: client(KEYS[KeeperKmsIdentity.GuardLifecycle]),
    [KeeperKmsIdentity.SettleTrigger]: client(KEYS[KeeperKmsIdentity.GuardLifecycle]),
    [KeeperKmsIdentity.WatchdogCanary]: client(KEYS[KeeperKmsIdentity.WatchdogCanary]),
  }
}

describe('M-02: canonical V4 fused identity is rejected by production default', () => {
  it('rejects GuardLifecycle/SettleTrigger fusion when no allowance is supplied', async () => {
    const result = await Effect.runPromise(Effect.either(assertDistinctIdentities(fusedClients())))

    expect(Either.isLeft(result)).toBe(true)
    if (Either.isLeft(result)) {
      expect(result.left.reason).toContain(KeeperKmsIdentity.GuardLifecycle)
      expect(result.left.reason).toContain(KeeperKmsIdentity.SettleTrigger)
    }
```

```
  })

  it('accepts only the intended fused pair when the existing allowance is supplied', async () => {
   const addresses = await Effect.runPromise(
    assertDistinctIdentities(fusedClients(), [
     [KeeperKmsIdentity.GuardLifecycle, KeeperKmsIdentity.SettleTrigger],
    ]),
   )

   expect(addresses[KeeperKmsIdentity.GuardLifecycle]).toBe(
    addresses[KeeperKmsIdentity.SettleTrigger],
   )
  })
 })
```

Run:

```
pnpm --filter @rini/keeper exec vitest run src/infrastructure/adapters/M02-v4-fused-identity.test.ts
```

Observed: 1 test file passed, 2 tests passed.

Recommendation

Use the canonical V4 identity model in the production composition. Permit only the `GuardLifecycle` / `SettleTrigger` fusion, route both duties through the same durable nonce sequence, and verify the resolved signer against the live Guard authority at startup and at hot adoption. Keep all unrelated identities distinct.

Team Response

Fixed.

M-03

MEDIUM FIXED

The Real Keeper Settlement Transport Cannot Execute Against Factory-Created V4 Stacks

Description

The `VaultStackFactory` enables and registry-binds a `RiniSettleModule` as the Safe's settlement module. `RiniSafeGuard` rejects every other module, and the frozen owner path cannot enable a new module after deployment.

The real keeper settlement adapter targets the removed Zodiac Roles topology instead. Startup requires `rolesModule` and `settleRoleKey` under `KEEPER_SETTLEMENT_ADAPTER=real`, and the adapter calls `execTransactionWithRole()` on that configured Roles module. A factory-created V4 Safe neither enables that module nor permits it through the module guard. The direct V4 settlement boundary exists in the keeper source (`v4/settlement-boundary.ts`), but it has zero production consumers. Production composition uses the Roles transport.

No configuration fixes this. Any address declared as `rolesModule` either fails Safe admission because the module is not enabled, or points at the settle module and calls a function that contract does not implement.

Location of Affected Code

File: `contracts/src/SafeSetupInitializer.sol`

```
self.enableModule(module);
self.setGuard(txGuard);
self.setModuleGuard(moduleGuard);
```

File: `contracts/src/RiniSafeGuard.sol`

```
SafeStackRegistry.StackBinding memory binding = registry.bindingOf(safe);
if (binding.vault == address(0)) revert SafeNotRegistered(safe);
// Only the stack's own settle boundary may drive the Safe. A module enabled by any other
// route is refused even before its target is examined.
if (module != binding.settleModule) revert UnknownModule(module, binding.settleModule);
```

File: `services/keeper/src/infrastructure/adapters/settle-transports.adapter.ts`

```
const data = encodeFunctionData({
  abi: ROLES_ABI,
  functionName: 'execTransactionWithRole',
  args: [address, On, inner, 0, roles.roleKey, true],
})
```

Impact

Real keeper deposit settlement, redemption settlement, and fee crystallization cannot execute against factory-created V4 stacks. The failure is detected and can trigger a custody hold, but affected requests remain unsettled until the transport is replaced. No theft or mispricing is claimed.

Proof of Concept

Save as `contracts/test/RiniM03IssuePoc.t.sol`:

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.26;

import {VaultStackFactoryTest} from "./unit/VaultStackFactory.t.sol";
import {Deployed} from "../src/VaultStackFactory.sol";
import {RiniSafeGuard} from "../src/RiniSafeGuard.sol";
import {SafeOperation} from "../src/interfaces/ISafeGuards.sol";
import {MockSafe} from "./mocks/VaultStackInfraMocks.sol";

contract FakeZodiacRolesModule {
  function forward(address safe, address vault, bytes calldata data) external returns (bool) {
    (bool ok, bytes memory ret) = safe.call(
      abi.encodeWithSignature(
        "execTransactionFromModule(address,uint256,bytes,uint8)",
        vault,
        uint256(0),
        data,
        uint8(0)
      )
    );
    if (!ok) {
      assembly {
        revert(add(ret, 0x20), mload(ret))
      }
    }
    return abi.decode(ret, (bool));
  }
}

contract RiniM03IssuePoc is VaultStackFactoryTest {
  function test_PoC_realRolesSettlementCannotComposeWithFactoryV4Stack() public {
    vm.prank(leader);
    Deployed memory out = factory.createStack(_config(1));
    FakeZodiacRolesModule roles = new FakeZodiacRolesModule();
    bytes memory settle = abi.encodeWithSignature("settleDeposit(uint256)", uint256(1));

    assertTrue(MockSafe(out.safe).isModuleEnabled(out.settleModule));
    assertFalse(MockSafe(out.safe).isModuleEnabled(address(roles)));

    vm.expectRevert(bytes("MockSafe: module not enabled"));
    roles.forward(out.safe, out.vault, settle);

    vm.prank(out.safe);
    vm.expectRevert(
      abi.encodeWithSelector(RiniSafeGuard.UnknownModule.selector, address(roles), out.settleModule)
    );
    safeGuard.checkModuleTransaction(
      out.vault,
      0,
```

```
        settle,
        SafeOperation.Call,
        address(roles)
      );
    }
}
```

Run:

```
cd contracts
forge test --match-path test/RiniM03IssuePoc.t.sol --match-test
test_PoC_realRolesSettlementCannotComposeWithFactoryV4Stack -vv
```

Observed: 1 test passed, 0 failed, 0 skipped.

Recommendation

For V4 vaults, call the registry-bound `RiniSettleModule.settleDeposit()` and `settleRedeem()` entry points directly. Remove the Zodiac Roles requirements from V4 startup, and verify the registry tuple, module runtime, Safe enablement, guard slots, and live settle authority before arming settlement.

Team Response

Fixed.

M-04

MEDIUM FIXED

A Proposed-Agent Identity Pin Can Repeatedly Block Steward-Owned Role Rotations

Description

The `CuratorGuard.isTradingAgent()` treats `proposedAgent` as a trading agent without checking the proposal's expiry. A vault leader sets that slot through `proposeAgentChange()`, and that function is the only one of the three propose families with no pending-proposal guard, so the leader can re-submit immediately after a guardian veto.

`VaultOwnerTimelock` revalidates protocol-role candidates at activation and rejects any address for which `isTradingAgent()` returns true. After the steward queues a valuation-manager, security-council, or sanctions-list rotation, the leader proposes the same address as the next trading agent. The matured steward operation reverts with `RoleCollision` and stays unexecuted.

Veto does not contain this. The guardian can veto the agent proposal, and the leader can immediately submit it again. The steward can queue a fresh operation with a different candidate, and the leader can pin that one too. An expired proposal stays in the predicate as well: the only expiry-cleanup path, `_clearExpiredAgentProposal()`, is called from `proposeVaultLeaderChange()`, which the same leader controls. The rotation family stays blocked for as long as the leader is hostile, and no role can rotate the leader against its will.

This finding assumes the vault leader is not intended to hold unilateral power to block steward-owned protocol-role rotations. The architecture documentation scopes the leader to vault-scoped owner changes, its own key rotation, and trading-agent rotation, and the safety model explicitly models a hostile leader. If the blocking power is intentional, it should be documented as such.

Location of Affected Code

File: `contracts/src/CuratorGuard.sol`

```
function isTradingAgent(address candidate) public view returns (bool) {
 return candidate != address(0)
    && (candidate == tradingAgent
     || candidate == activeAgent
     || candidate == pendingAgent
     || candidate == proposedAgent);
}
```

```
function proposeAgentChange(address agent) external onlyVaultLeader {
  _requireValidAgent(agent);
  proposedAgent = agent;
  proposedAgentEpoch = leaderEpoch;
  proposedAgentEta = block.timestamp + TIMELOCK_DELAY;
  emit AgentChangeProposed(agent, proposedAgentEta, proposedAgentEta + OP_EXPIRATION, leaderEpoch);
}
```

File: `contracts/src/VaultOwnerTimelock.sol`

```
 if (
  candidate == _leader() || guard.isTradingAgent(candidate) || guard.isProtocolIdentity(candidate)
    || _isVaultRole(candidate)
 ) {
   revert RoleCollision(candidate);
 }
```

Impact

A hostile or compromised vault leader can block steward-owned rotations of security-critical protocol identities indefinitely, at the cost of one `proposeAgentChange()` transaction per rotation attempt. No funds move directly. Incident response and governance recovery stay unavailable for the affected vault for as long as the block lasts.

The same expiry-blindness affects the non-adversarial case as well; see L-14 (Expired Role Proposals Can Permanently Block Subsequent Role Rotations), which requires a separate fix covering all three proposal families.

Proof of Concept

Save as `contracts/test/unit/RiniM04TimelockIssuePoc.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {VaultOwnerTimelock} from "../../src/VaultOwnerTimelock.sol";
import {
  TimelockLeaderSeatMock,
  TimelockStackRegistryMock,
  TimelockVaultMock
} from "./VaultOwnerTimelock.t.sol";

/// Reproduces activation-time role collision after a steward operation is queued.
contract AuditPocTimelockLegTest is Test {
  address internal steward = makeAddr("steward");
  address internal guardian = makeAddr("guardian");
  address internal leader = makeAddr("leader");

  TimelockVaultMock internal vault;
  TimelockStackRegistryMock internal registry;
  TimelockLeaderSeatMock internal guard;
  VaultOwnerTimelock internal timelock;

  function setUp() public {
    address guardSafe = makeAddr("guardSafe");
    vault = new TimelockVaultMock(address(this), guardSafe);
    registry = new TimelockStackRegistryMock();
    guard = new TimelockLeaderSeatMock(leader, guardSafe, address(registry), makeAddr("keeper"), guardian, steward);
    registry.bind(guard.safe(), address(vault), address(guard), makeAddr("module"));
    timelock =
      new VaultOwnerTimelock(address(vault), steward, guardian, address(guard), address(registry), address(0));
    vault.transferOwnership(address(timelock));
    timelock.acceptOwnership();
```

```
    }

    /// The leader pins the steward's already-queued target; the matured activation is dead.
    function test_poc01_leaderPinKillsStewardValuationManagerRotation() public {
        address candidate = makeAddr("newValuationManager");

        // 1. Steward queues the rotation. Validation passes at propose time.
        vm.prank(steward);
        bytes32 opId = timelock.proposeUpdateValuationManager(candidate);
        assertTrue(timelock.opEta(opId) != 0, "op should be queued");

        // 2. The op is now public, with its target and eta readable on chain. The leader pins that
        //    exact address as proposedAgent - no front-running, no race, ~50k gas on the real Guard.
        guard.setTradingAgent(candidate, true);

        // 3. The timelock matures. Activation is permissionless, and it re-validates.
        vm.warp(vm.getBlockTimestamp() + 3 days);
        vm.expectRevert(abi.encodeWithSelector(VaultOwnerTimelock.RoleCollision.selector, candidate));
        timelock.activateUpdateValuationManager(candidate, 0);

        // 4. The op is neither activated nor cleared - it is stuck.
        assertTrue(timelock.opEta(opId) != 0, "op remains queued but permanently unactivatable");
    }

    /// Same lever, the other two steward-owned families.
    function test_poc01_leaderPinKillsSecurityCouncilAndSanctionsRotations() public {
        address council = makeAddr("newSecurityCouncil");
        vm.prank(steward);
        timelock.proposeUpdateSecurityCouncil(council);
        guard.setTradingAgent(council, true);
        vm.warp(vm.getBlockTimestamp() + 3 days);
        vm.expectRevert(abi.encodeWithSelector(VaultOwnerTimelock.RoleCollision.selector, council));
        timelock.activateUpdateSecurityCouncil(council, 0);
    }

    /// Control: without the pin, the identical rotation activates cleanly.
    function test_poc01_control_rotationSucceedsWithoutThePin() public {
        address candidate = makeAddr("newValuationManager");
        vm.prank(steward);
        timelock.proposeUpdateValuationManager(candidate);
        vm.warp(vm.getBlockTimestamp() + 3 days);
        timelock.activateUpdateValuationManager(candidate, 0);
        // No revert: the ONLY difference from the first test is the leader's pin.
    }
}
```

Run:

```
cd contracts
forge test --match-path test/unit/RiniM04TimelockIssuePoc.t.sol -vv
```

Observed: 3 tests passed, 0 failed, 0 skipped.

The production `CuratorGuard` behavior was separately exercised with five passing tests, including proposal expiry and immediate re-pinning after a guardian veto.

Recommendation

Use separate predicates for active trading authority and pending nominations. Cross-contract role-collision checks must not treat a merely proposed or expired agent as an active trading agent. Add explicit pending-proposal and permissionless expiry-cleanup semantics to the agent family, and test activation of a steward operation whose target is nominated as an agent after queueing.

Team Response

Fixed.

M-05

MEDIUM FIXED

The Real Bridge Driver Reads `CuratorGuard` Through a Pre-V4 Surface

Description

The `BridgeDriverPort.readState()` is the first thing every bridge-drive tick does, before it can inspect or advance any flow. It reads ten `CuratorGuard` getters concurrently.

Four of them do not exist on the audited `contracts/src/CuratorGuard.sol`: `usdcBalanceBeforeBridge()`, `balanceBeforeReturn()`, `coreBalanceBeforeReturn()` and `bridgeCap()`. `CuratorGuard` has no `fallback` or `receive`, so an unknown selector makes the low-level call itself fail. Each of the four reads reverts, and the whole `Effect.all` fails.

The control that exists to prevent exactly this does not check contract shape. `verifiedGuard` gates on `isCuratorGuardBridgeAbiVerified(opts.chainId)`, which is chain identity and nothing else: the function returns true when the chain id equals HyperEVM mainnet. Its own comment records that the verification it stands for was a codehash round-trip against a canary Guard on chain 999, a different contract generation. The gate answers whether this is mainnet. It never answers whether the Guard the keeper resolved publishes these selectors.

The protocol's own safety documentation independently confirms that V4 has no `bridgeCap`. This is the same family as M-03, a keeper transport bound to a removed contract generation, but a different file, port, and remediation: fixing the settlement transport leaves `readState()` reverting, and vice versa.

Location of Affected Code

File: `contracts/src/CuratorGuard.sol` – the complete flow-state surface. None of the four requested names is declared anywhere in the file:

```
uint256 public currentBridged;
uint256 public exposureEpoch;
uint256 public sweptUnreleased;

uint256 public nextFlowId = 1;
mapping(uint256 flowId => FlowRecord record) public flows;

uint256 public pendingBridgeFlowId;
uint256 public pendingBridgeAmount;
uint256 public coreBalanceBeforeBridge;
uint256 public pendingBridgeInitiatedAt;

uint256 public pendingReturnFlowId;
uint256 public pendingReturnAmount;
uint256 public pendingReturnInitiatedAt;
```

File: `services/keeper/src/infrastructure/adapters/curator-guard-bridge-driver.adapter.ts`

```
const [
  pendingBridgeAmount,
  coreBalanceBeforeBridge,
  usdcBalanceBeforeBridge,
  pendingBridgeInitiatedAt,
  pendingReturnAmount,
  balanceBeforeReturn,
  coreBalanceBeforeReturn,
  pendingReturnInitiatedAt,
  currentBridged,
  bridgeCap,
] = yield* Effect.all([
  [
    readU256(opts.read, address, 'pendingBridgeAmount'),
    readU256(opts.read, address, 'coreBalanceBeforeBridge'),
    readU256(opts.read, address, 'usdcBalanceBeforeBridge'),
    readU256(opts.read, address, 'pendingBridgeInitiatedAt'),
    readU256(opts.read, address, 'pendingReturnAmount'),
    readU256(opts.read, address, 'balanceBeforeReturn'),
    readU256(opts.read, address, 'coreBalanceBeforeReturn'),
    readU256(opts.read, address, 'pendingReturnInitiatedAt'),
    readU256(opts.read, address, 'currentBridged'),
    readU256(opts.read, address, 'bridgeCap'),
  ],
```

```
  { concurrency: 'unbounded' },
]
```

The gate that is supposed to make this safe, in the same file:

```
/** Fail closed when the ABI is unverified and we are not explicitly allowing it (G2). */
const verifiedGuard = (leg: Leg | 'read-state'): Effect.Effect<void, BridgeDriveError> => {
 if (!isCuratorGuardBridgeAbiVerified(opts.chainId) && !allowUnverified) {
```

A related call site shares the same root premise through a different port. File: `services/keeper/src/infrastructure/adapters/guard-bridged-read.adapter.ts`

```
const state = yield* Effect.all(
 {
  currentBridged: readAmount(address, 'currentBridged', blockNumber),
  bridgeCap: readAmount(address, 'bridgeCap', blockNumber),
  pendingBridgeAmount: readAmount(address, 'pendingBridgeAmount', blockNumber),
  pendingReturnAmount: readAmount(address, 'pendingReturnAmount', blockNumber),
 },
 { concurrency: 'unbounded' },
)
```

Impact

For every operated vault whose keeper is armed with `KEEPER_BRIDGE_DRIVE_ADAPTER=real` against the audited V4 `CuratorGuard`, the autonomous EVM-to-Core bridge driver is dead on arrival. `bridgeDriveJob()` is scheduled, fans out over `operatedVaults`, and the drive use-case's first act for every vault is `readState()`, which cannot complete. No allocation and no return is ever initiated.

In-flight flows cannot be advanced or confirmed either. `confirmBridge()`, `initiateReturn()`, and `recordBridgeBack()` all sit downstream of the failed read, as does the escalation path. A pending flow ages toward the contract's one-hour timeouts with no automated party able to act. Recovery falls to guardian timeout paths, and the Guard's single-flow lock stays engaged for the duration.

The failure never pages an operator. The observable symptom is one warning line per cadence, forever: no notifier, no custody hold, no consecutive-failure escalation. The outage lasts until the keeper's read surface is corrected or an operator runs a manual bridge-leg recovery. No value is at risk, because the driver fails before submitting anything. Depositor exits that depend on repatriated Core liquidity are delayed, not lost.

Canonical documentation asserts no V4 deployment, so present impact is deployment-conditional. The code path is live the moment the real adapter is armed.

Proof of Concept

Save as `contracts/test/RiniM05IssuePoc.t.sol`:

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.28;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {CuratorGuard} from "../src/CuratorGuard.sol";
import {CuratorGuardTest} from "./unit/CuratorGuard.t.sol";

/// @dev A contract that "answers everything" — the shape of the keeper's name-based fake read
///      transports. The SAME ten-getter read sequence passes against it, which is mechanically
///      why `curator-guard-bridge-driver.adapter.test.ts` stays green (35/35) against a defect
///      that makes the real adapter unusable.
contract AnswerEverythingGuard {
  fallback(bytes calldata) external payable returns (bytes memory) {
    return abi.encode(uint256(1));
  }

  receive() external payable {}
}

/**
 * M-05 PoC: the real keeper bridge driver's `readState` cannot complete against the audited
 * CuratorGuard. `curator-guard-bridge-driver.adapter.ts` concurrently reads ten getters; four of
 * them — usdcBalanceBeforeBridge(), balanceBeforeReturn(), coreBalanceBeforeReturn() and
 * bridgeCap() — do not exist on `contracts/src/CuratorGuard.sol`, and the contract has no
 * `fallback`/`receive`, so an unknown selector makes the low-level call itself fail and the
 * whole `Effect.all` reverts: both autonomous bridge legs are dead before they start.
 *
 * This PoC deploys the AUDITED CuratorGuard (not a mock) with the same constructor wiring the
 * unit harness uses, replays the adapter's exact ten-getter sequence via the same low-level
 * read shape a viem `readContract` uses, and adds the negative control that explains why the
 * keeper's own unit suite stays green: a fake whose fallback answers every getter passes.
 */
contract RiniM05IssuePoc is CuratorGuardTest {
  /// @dev The EXACT ten getters, in the adapter's order
  ///      (curator-guard-bridge-driver.adapter.ts, `readState`'s Effect.all array).
  string[10] internal DRIVER_GETTERS = [
    "pendingBridgeAmount()",
    "coreBalanceBeforeBridge()",
    "usdcBalanceBeforeBridge()",
    "pendingBridgeInitiatedAt()",
    "pendingReturnAmount()",
    "balanceBeforeReturn()",
    "coreBalanceBeforeReturn()",
    "pendingReturnInitiatedAt()",
    "currentBridged()",
```

```
    "bridgeCap()"
  ];

  /// @dev The four selectors the audited contract does not declare anywhere.
  string[4] internal ABSENT_GETTERS = [
    "usdcBalanceBeforeBridge()",
    "balanceBeforeReturn()",
    "coreBalanceBeforeReturn()",
    "bridgeCap()"
  ];

  /// @dev The six getters the audited contract DOES declare (its complete public flow state).
  string[6] internal PRESENT_GETTERS = [
    "pendingBridgeAmount()",
    "coreBalanceBeforeBridge()",
    "pendingBridgeInitiatedAt()",
    "pendingReturnAmount()",
    "pendingReturnInitiatedAt()",
    "currentBridged()"
  ];

  /// @dev One viem-shaped readContract: a plain staticcall of the 4-byte selector.
  function _readU256(address target, string memory signature)
    internal
    view
    returns (bool ok, uint256 value)
  {
    (bool success, bytes memory ret) = target.staticcall(abi.encodeWithSignature(signature));
    if (success && ret.length >= 32) {
      return (true, abi.decode(ret, (uint256)));
    }
    return (false, 0);
  }

  /// @dev The audited CuratorGuard itself — same constructor wiring as the unit harness,
  ///      no mock, no harness subclass.
  function _auditedGuard() internal returns (CuratorGuard) {
    CuratorGuard.AuthorityConfig memory cfg = CuratorGuard.AuthorityConfig({
      stackRegistry: address(authority),
      vaultLeader: address(uint160(uint256(keccak256("HarnessGuard.vaultLeader")))),
      protocolIdentities: new address[](0)
    });
    return new CuratorGuard(
      IERC20(address(usdc)),
      safe,
      settleTrigger,
      guardian,
      steward,
      builder,
```

```
        BUILDER_RATE,
        tradingAgent,
        cfg
    );
}

function test_PoC_realDriverReadStateCannotCompleteAgainstAuditedCuratorGuard() public {
    CuratorGuard guard = _auditedGuard();

    // 1. The adapter's readState: read ALL ten getters. The batch is dead the moment ANY
    //    one of them fails, so simulate the Effect.all and record the first failure.
    uint256 servedBeforeFailure = 0;
    string memory firstAbsentSeen = "";
    for (uint256 i; i < DRIVER_GETTERS.length; ++i) {
        (bool ok,) = _readU256(address(guard), DRIVER_GETTERS[i]);
        if (!ok) {
            firstAbsentSeen = DRIVER_GETTERS[i];
            break;
        }
        servedBeforeFailure += 1;
    }
    assertTrue(bytes(firstAbsentSeen).length != 0, "readState unexpectedly completed");
    assertEq(servedBeforeFailure, 2, "first absent getter is usdcBalanceBeforeBridge");

    // 2. Each of the four removed selectors fails INDIVIDUALLY (no fallback, no receive:
    //    the low-level call itself returns ok=false, exactly what readU256 surfaces as a
    //    revert and what makes the whole Effect.all fail).
    for (uint256 i; i < ABSENT_GETTERS.length; ++i) {
        (bool ok,) = _readU256(address(guard), ABSENT_GETTERS[i]);
        assertFalse(ok, string.concat("getter should not exist: ", ABSENT_GETTERS[i]));
    }

    // 3. Positive control: every getter the contract DOES publish is served, so the fault
    //    is specific to the four removed names, not an artifact of this harness.
    for (uint256 i; i < PRESENT_GETTERS.length; ++i) {
        (bool ok,) = _readU256(address(guard), PRESENT_GETTERS[i]);
        assertTrue(ok, string.concat("present getter should be served: ", PRESENT_GETTERS[i]));
    }
}

function test_PoC_negativeControl_answerEverythingFakePassesTheSameReads() public {
    AnswerEverythingGuard fake = new AnswerEverythingGuard();

    // The SAME ten-getter sequence succeeds against an ABI-blind fake: this is why the
    // keeper's unit suite (fake transports answering by requested name) cannot detect the
    // drift, and why the real composition is the one that breaks.
    for (uint256 i; i < DRIVER_GETTERS.length; ++i) {
        (bool ok,) = _readU256(address(fake), DRIVER_GETTERS[i]);
        assertTrue(ok, "answer-everything fake serves every getter, including absent ones");
```

```
    }
  }
}
```

Run:

```
cd contracts
forge test --match-path test/RiniM05IssuePoc.t.sol --match-test test_PoC -vv
```

Observed: 2 passed, 0 failed, 0 skipped.

```
[PASS] test_PoC_negativeControl_answerEverythingFakePassesTheSameReads() (gas: 87237)
[PASS] test_PoC_realDriverReadStateCannotCompleteAgainstAuditedCuratorGuard() (gas: 3996810)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 9.38ms
```

The failing test is the finding. The audited `CuratorGuard` serves exactly two of the adapter's ten getters before `usdcBalanceBeforeBridge()` kills the batch. Each of the four removed selectors fails individually, and all six getters the contract does publish are served, so the fault is specific to the removed names. The negative control is the blind spot: an ABI-blind fake that answers every getter passes the same ten reads, which is mechanically why the existing adapter suite stays green (35 of 35) against a defect that makes the real adapter unusable. `forge inspect CuratorGuard methods` corroborates: 75 methods, zero matches for the four getters.

Recommendation

Regenerate every keeper Guard read surface – the bridge driver's `readState()` and `guard-bridged-read.adapter.ts`'s `readGuardState()` – from the compiled current `CuratorGuard` ABI. Delete `bridgeCap` and the three removed snapshot fields from `GuardBridgeState`, `GuardLiquidityState`, and `guard-bridged-read.port.ts` instead of defaulting them. Redesign allocation authorization around V4's documented no-cap model rather than synthesizing a replacement `bridgeCap` value.

Replace the chain-id-only `isCuratorGuardBridgeAbiVerified` gate with a shape check: at boot and at hot adoption, verify the resolved Guard's codehash, or probe every required selector, against the compiled in-scope ABI, and refuse to schedule money-path jobs on mismatch. Add a build gate that diffs every production adapter getter and transaction selector against the compiled in-scope ABI, which prevents the same stale-ABI class from recurring across production adapter surfaces.

Replace the answer-by-name fake transports in the adapter tests with an artifact-derived strict transport, and add one real-contract integration test against factory-created V4 bytecode. Finally, give a repeatedly failing money-path job an escalation path. A warning line per cadence is not operator notification.

Team Response

Fixed.

L-01

LOW FIXED

Whitelist-Mode Stacks Ship With Unusable Safe Claim-on-Behalf Helpers

Description

`RiniSafeGuard` permits the Safe to call Lagoon's batch claim helpers:

- `claimSharesOnBehalf(address[])`
- `claimAssetsOnBehalf(address[])`

In whitelist mode, Lagoon checks that the caller is allowed. The caller is the Safe, but `VaultStackFactory` does not whitelist the Safe at genesis: it sets `superOperator` to zero and adds only `cfg.feeReceiver` to the whitelist.

As a result, both Safe claim-on-behalf helpers revert on a normal factory-created whitelist-mode stack.

Location of Affected Code

File: `contracts/src/VaultStackFactory.sol`

```
// Only the fee receiver is added during whitelist-mode genesis.
vault.addToWhitelist(whitelist);
```

File: `contracts/src/RiniSafeGuard.sol`

```
// The Safe is permitted to call both helpers.
if (selector == ILagoonVault.claimSharesOnBehalf.selector) return 0;
if (selector == ILagoonVault.claimAssetsOnBehalf.selector) return 0;
```

Impact

The Safe cannot batch-claim settled shares or assets for users in whitelist mode until the whitelist manager separately adds the Safe.

Recommendation

When creating a whitelist-mode stack, add the Safe to the whitelist at genesis. Alternatively, remove these two helper selectors from the Safe policy if they are intentionally unsupported in whitelist mode.

Team Response

Fixed.

L-02

LOW FIXED

Minting the Timelock as Whitelist Manager Blocks Steward Role Rotations

Description

The `VaultStackFactory.createStack()` accepts the leader-supplied `whitelistManager` before it deploys the stack's `VaultOwnerTimelock`. The timelock address is deterministic from `saltNonce`, so a leader can set `cfg.whitelistManager` to that future address.

The stack then mints with the timelock as its own whitelist manager. Later, the timelock identifies the current whitelist manager by probing `addToWhitelist([])` and expecting an unauthorized-call revert. As the manager, the timelock's probe succeeds instead, so the probe reverts with `VaultRoleProbeFailed`.

This prevents the steward-controlled role-rotation path from completing. The vault leader is the only role that can repair the configuration by rotating the whitelist manager after the timelock delay.

Location of Affected Code

File: `contracts/src/VaultStackFactory.sol`

```
// The leader chooses this before the timelock is deployed.
vault.updateWhitelistManager(cfg.whitelistManager);
```

File: `contracts/src/VaultOwnerTimelock.sol`

```
(bool ok, bytes memory data) = vault.staticcall(callData);
if (ok || data.length != 36 || actualSelector != expectedSelector) {
  revert VaultRoleProbeFailed(expectedSelector);
}
```

Impact

The steward cannot rotate the valuation manager or security council for the affected stack until the leader repairs the whitelist-manager seat.

This is one instance of a broader validation gap in the factory's whitelist-manager admission; see L-13 (Whitelist Administration Can Be Disabled Because Factory Validation Accepts Inert Managers) for the other derived stack addresses that are also accepted.

Proof of Concept

1. Calculate the deterministic timelock address `T` for a chosen `saltNonce`.
2. Call `createStack()` with that nonce and set `cfg.whitelistManager = T`.
3. The stack mints successfully and `T` becomes the vault's whitelist manager.
4. A steward role-rotation call reaches the timelock's whitelist-manager probe and reverts with `VaultRoleProbeFailed`.
5. Only the current leader can queue a whitelist-manager change to repair the stack.

Recommendation

Reject the predicted timelock address as the mint-time whitelist manager.

```
if (cfg.whitelistManager == predictedTimelock) {
  revert ProtocolIdentityCollision(3, cfg.whitelistManager);
}
```

Team Response

Fixed.

L-03

LOW FIXED

Guard-Lifecycle Nonce Monitor Does Not Re-Arm After Recovery

Description

The keeper monitors the shared GuardLifecycle signer's nonce state and sends a critical alert when the durable nonce frontier does not match the chain state.

The alert key contains only the chain, signer, and current divergence:

```
guard_lifecycle_nonce_divergence:<chainId>:<signerAddress>:<divergence>
```

When the signer becomes healthy again, the monitor simply returns. It does not clear or re-arm the previous alert key. The production SNS adapter remembers successfully delivered keys without a TTL, only evicting an old key after 2,000 other distinct keys.

Consequently, a later independent incident that recreates the same divergence is logged but does not send another critical alert.

Including `divergence` in the key only re-alerts when the gap changes during an incident. It does not distinguish two separate incidents divided by a healthy period.

Location of Affected Code

File: `services/keeper/src/application/keeper-schedule.ts`

```
const verdict = classifyNonceHealth(snapshot)
if (verdict.healthy) return

yield* notify.alert({
 severity: 'critical',
 kind: 'guard_lifecycle_nonce_divergence',
 dedupeKey:
  `guard_lifecycle_nonce_divergence:${identity.chainId}:${identity.signerAddress}:${verdict.divergence}`,
})
```

File: `services/keeper/src/infrastructure/adapters/sns-notification.adapter.ts`

```
const seenAlertKeys = new Map<string, true>()

if (key !== undefined && seenAlertKeys.has(key)) return Effect.void

// The key is retained after a successful publish.
if (key !== undefined) rememberBounded(seenAlertKeys, key)
```

Impact

A new nonce-integrity incident can receive no critical page after an earlier same-sized incident was resolved. Operators may not discover that the privileged shared writer is unhealthy until another Guard operation fails.

The affected identity can serve fleet-wide bridge, perp-hop, agent-rotation, builder-fee, and settlement operations, so losing the recurrence alert can delay response across multiple vaults.

Proof of Concept

1. A dead reservation at nonce `0` leaves `nextNonce = 1`, `onChainNonce = 0`, and no live reservations. The divergence is `+1`, and the keeper sends the first critical alert.
2. The reservation is backfilled and mined. The monitor returns to a healthy state.
3. Later, a separate dead reservation at nonce `1` leaves `nextNonce = 2`, `onChainNonce = 1`, and no live reservations. The new divergence is again `+1`.
4. The monitor detects and logs the second incident, but the notifier suppresses its page because the first incident already stored the identical key:

```
alert:guard_lifecycle_nonce_divergence:<chainId>:<signerAddress>:1
```

Thus, two independent incidents produce only one critical notification within the same keeper process.

Recommendation

Track nonce-health episodes and include an episode identifier in the alert key:

```
guard_lifecycle_nonce_divergence:<chainId>:<signerAddress>:<episode>:<divergence>
```

Increment the episode on every `healthy -> unhealthy` transition. This preserves deduplication while an incident remains active but allows a recovered-and-repeated incident to page again.

Add a regression test proving that `unhealthy(+1) -> healthy -> unhealthy(+1)` sends two critical alerts.

Team Response

Fixed.

L-04

LOW FIXED

Hot-Adopted Liquidity Policies Bypass the Autonomous-Liquidity Admission Interlock

Description

The keeper validates autonomous-liquidity configuration only against the vaults present at startup.

When `bridgeDrive=real` and a startup vault has `liquidityPolicy`, `buildRealAdapterDeps()` requires the `bridge`, `bridgeIndex`, `vault`, and `hyperCore` seams to also be real. This prevents real bridge transactions from being driven with mock safety inputs.

The operated-vault registry can add vaults without restarting the keeper. However, its admission validator checks only builder-fee configuration. A hot-adopted vault containing `liquidityPolicy` is therefore published without applying the autonomous-liquidity seam check.

The keeper rebuilds its liquidity runtimes from the live vault list on every tick. Consequently, the newly adopted vault immediately begins using the liquidity-policy and real bridge-driver jobs under a configuration that would have been rejected at startup.

Location of Affected Code

File: `services/keeper/src/infrastructure/build-deps.ts`

```
const hasLiquidityWriter = config.operatedVaults.some(
 (vault) => vault.liquidityPolicy !== undefined,
)

if (hasLiquidityWriter && config.adapters.bridgeDrive === 'real') {
 // Requires bridge, bridgeIndex, vault, and hyperCore to be real.
}
```

File: `services/keeper/src/application/commissioning.ts`

```
admissionReason = admissionFailure(parsedEntry.vault)
...
merged.push(parsedEntry.vault)
```

File: `services/keeper/src/application/keeper-schedule.ts`

```
const liquidityVaultsProvider = () =>
 makeLiquidityVaultRuntimes(config.operatedVaults, config.adapters.bridgeDrive)
```

The startup check is at `services/keeper/src/infrastructure/build-deps.ts:472-496`. The unguarded registry admission is at `services/keeper/src/application/commissioning.ts:197-200` and `:315-336`. Runtime hot activation occurs at `services/keeper/src/application/keeper-schedule.ts:2047-2062`.

Impact

A hot-adopted policy can drive real bridge transactions using mock safety inputs.

For example, if `bridge=mock`, one standing allocation request can be issued repeatedly. The mock reconciler never observes that the first allocation arrived, so the request is not closed and the keeper transfers the same amount again on later ticks. Safe USDC can continue moving to Core until the exposure cap or Safe balance stops it.

Proof of Concept

1. Start the keeper with registry adoption enabled, `bridgeDrive=real`, `bridgeIndex=real`, `vault=real`, `hyperCore=real`, and `bridge=mock`. Ensure no startup vault has `liquidityPolicy`, so the boot interlock does not execute.
2. Hot-adopt a valid vault configuration containing `liquidityPolicy` and create a standing allocation request for `X` USDC.
3. `syncOperatedVaultsJob()` publishes the vault because its admission validator does not check the liquidity seam matrix.
4. On the next liquidity tick, the mock reconciler reports `allocateArrived=false` and `readInFlight=0`. `standingAllocationWriter()` writes an allocation intent for `X`.
5. The real bridge driver submits and confirms the allocation.
6. On a later tick, the standing request is still open because the mock reconciler cannot observe the recorded arrival. The keeper writes and executes another allocation for `X`.
7. The process repeats until the Guard exposure cap or the Safe's USDC balance prevents another allocation.

The same configuration is rejected when the policy vault is present at startup.

Recommendation

Apply the startup autonomous-liquidity interlock during registry admission, before adding the vault to the live list.

When a hot-adopted vault contains `liquidityPolicy` and `bridgeDrive=real`, require `bridge`, `bridgeIndex`, `vault`, and `hyperCore` to all be real. Keep `liquidityPolicy` with `bridgeDrive=mock` permitted because it cannot submit real bridge transactions.

Factor both paths through one shared validator and add tests proving that every startup-rejected seam combination is also rejected during hot adoption.

Team Response

Fixed.

L-05

LOW FIXED

A Stale Liquidity-Policy Worker Can Clear a Newer Shortfall Episode

Description

The scheduled `liquidity-policy` job intentionally runs outside `runKeeperJob()` because its database writes are treated as harmless plain overwrites. The scheduler's singleton lease reduces overlap between keeper instances, but it does not fence policy writes with the lease generation. After the lease expires, a successor can acquire it before the predecessor's next heartbeat observes the loss and interrupts its job fibre.

During that takeover window, both workers can finish decisions derived from different snapshots. A stale predecessor that observed a sufficient buffer can write a zero return intent and clear `liquidity_shortfall_since` after the successor has written a positive return intent and established or preserved the current shortfall episode. The successor's newer return request is then suppressed until policy runs again, while the durable SLA anchor is lost or restarted.

The reverse ordering is also unsafe. A stale predecessor that observed a shortfall can overwrite a successor's newer zero decision with a positive return intent and can re-engage the shortfall anchor after the condition has resolved. The return consumer applies custody, Core-balance, and exposure bounds, but it does not recompute the current liquidity shortfall before consuming a return intent. This can therefore request unnecessary custody-contained repatriation, although it cannot escape custody or exceed confirmed bridged exposure.

The repository API makes the race structural. Return intent is last-write-wins, while `clearLiquidityShortfall()` receives only the vault ID. Neither operation carries an observation revision, expected timestamp, or lease fencing token. A two-client PostgreSQL reproduction confirms that a stale clear is accepted after lease takeover.

Location of Affected Code

File: `services/keeper/src/application/keeper-schedule.ts`

```
* LIQUIDITY-POLICY. The `liquidity-policy` slot dispatches the real
* {@link liquidityPolicy} use-case (ADR-0005 §1) per operated vault. It is NOT routed
* through `runKeeperJob`: unlike the settle jobs (which call real
* `settleDeposits`/`settleRedeems` mutations), liquidity-policy is non-authoritative —
* it sizes the buffer, alerts/notifies (both dedupe-keyed → idempotent), and WRITES its
* decision as a durable return-intent (`RepositoryPort.vaults.setReturnIntent`) — so a
* doubled tick is harmless (plain overwrite) and needs no per-run lease. It never drives
* or confirms the bridge itself; `bridge-drive` below is the sole consumer of that intent.
```

File: services/keeper/src/application/liquidity-policy.ts

```
if (shortfall === On) {
 yield* effect(() => repo.vaults.setReturnIntent(config.vaultId, On))
 yield* effect(() => repo.vaults.clearLiquidityShortfall(config.vaultId))
 return {
  status: 'skipped',
  reason: 'buffer_sufficient',
  recoveryProven: fullyRepatriatedRecovery,
 } as LiquidityPolicyOutcome
}
```

```
yield* effect(() => repo.vaults.setReturnIntent(config.vaultId, planned))

if (capBounded) {
 // Ladder step ≥ 2: the shortfall cannot be met within the risk budget. Anchor the SLA
 // clock — idempotent-preserve (mirrors setCustodyHold) so a shortfall persisting across
 // many ticks keeps the SAME shortfall-since instant instead of resetting it every tick.
 yield* effect(() => repo.vaults.setLiquidityShortfall(config.vaultId, new Date(nowMs)))
```

File: services/keeper/src/domain/ports/repository.port.ts

```
 setReturnIntent: (id: VaultId, amount: UsdcAmount) => Effect.Effect<void, RepositoryError>
```

```
 setLiquidityShortfall: (id: VaultId, now: Date) => Effect.Effect<void, RepositoryError>
 clearLiquidityShortfall: (id: VaultId) => Effect.Effect<void, RepositoryError>
```

Impact

A stale zero decision can erase a newer bounded Core-to-EVM return request for up to the next liquidity-policy cadence, one hour by default. A stale clear can discard the current shortfall anchor. For a newly established episode, the SLA deadline shifts by up to one cadence; if the newer worker was preserving an older continuing episode, the elapsed history can be lost and the critical escalation delayed by that lost age plus the next cadence.

The opposite ordering can persist a stale positive return request or a stale shortfall episode after the buffer becomes sufficient. Existing bridge checks keep any resulting return within custody, confirmed Core spot liquidity, and current bridged exposure. No NAV manipulation, access-control bypass, bridge-cap bypass, or direct fund loss is established.

Recommendation

Fence the complete per-vault liquidity-policy decision at commit time. Assign each evaluation a monotonic policy revision or use the current singleton-lease generation as a fencing token, then atomically update the return intent and shortfall anchor only if that token is still current. A predecessor's commit must affect zero rows after takeover.

Do not fix only `clearLiquidityShortfall()`: compare-and-clear on the anchor would leave the return-intent overwrite race intact. Add a two-client PostgreSQL integration test covering both adverse directions and a negative control:

- successor positive intent and active anchor, followed by predecessor stale zero and clear;
- successor zero and cleared anchor, followed by predecessor stale positive intent and set;
- predecessor commit before successor commit, proving the current decision wins normally.

Team Response

Fixed.

L-06

LOW FIXED

Commissioning Can Mutate a Foreign-Chain Registry Row

Description

The `syncOperatedVaultsJob()` receives the keeper's `chainId` and rejects adopted rows registered for another chain. `commissioningJob()`, which drives the earlier `secured -> core_seeded` transition, does not receive or check a chain id. It processes every adopted `secured` row returned by the registry.

If a keeper is pointed at a shared authoritative database, a wrong-environment restore, or another database containing adopted rows from multiple chains, it can evaluate a foreign row using its own HyperCore connection and then mutate that row. A failed read can park the row as `stuck`, while a successful read can incorrectly advance it to `core_seeded`.

Location of Affected Code

File: `services/keeper/src/application/commissioning.ts`

The hot-adoption job explicitly takes and enforces `keeperChainId`:

```
if (row.chainId !== keeperChainId) {
 continue
}
```

`commissioningJob` has only three parameters – interval, fill window, and stuck timeout – and filters rows solely by status:

```
const secured = rows.filter((row) => row.commissioningStatus === 'secured')

yield* coreRead.getSpotBalance(coreAccount, vault.usdcTokenId).pipe(
 Effect.flatMap(() => registry.transition(row.vaultId, 'secured', 'core_seeded')),
 Effect.catchTag('CoreReadError', (e) =>
  Effect.gen(function* () {
   const age = nowMs - row.lastTransitionAt
   if (age < stuckTimeoutMs) return

   const reason = `core-seed postcondition unverified: ${e.reason}`
   yield* registry.markStuck(row.vaultId, reason)
  }),
 ),
)
```

File: `services/keeper/src/application/keeper-schedule.ts`

```
syncOperatedVaultsJob(
 config.operatedVaultsRegistry.syncIntervalMs,
 operatedVaultsHandle,
 envFallback,
 config.cadences.adlWatchMs,
 config.chainId,
 // ...
)

commissioningJob(
 config.operatedVaultsRegistry.commissioningIntervalMs,
 config.cadences.adlWatchMs,
 config.operatedVaultsRegistry.stuckTimeoutMs,
)
```

The first job receives `config.chainId`; the commissioning job does not. Its transition and `markStuck` calls also identify the row without a chain-id condition.

Impact

A wrong-chain keeper can either block a foreign vault's automated commissioning by marking it `stuck`, or falsely mark its Core seed as verified. The correct-chain keeper will not repeat the skipped verification, and recovering a `stuck` row requires operator intervention.

Proof of Concept

1. Add an adopted `secured` registry row with `chainId = 999` to a database used by a chain-998 keeper.
2. Set `lastTransitionAt` far enough in the past to exceed the commissioning timeout.
3. Run `commissioningJob()` while the chain-998 Core adapter returns a read failure.
4. The job selects the chain-999 row and changes its status to `stuck`.

If the wrong-chain Core read succeeds instead, step 4 changes the row to `core_seeded`. Running `syncOperatedVaultsJob()` with the same row does not have this behavior because that job rejects the chain-id mismatch.

Recommendation

Pass `config.chainId` into `commissioningJob()` and skip or alert on rows for which `row.chainId !== keeperChainId`, using the same policy as `syncOperatedVaultsJob()`.

Team Response

Fixed.

L-07

LOW FIXED

Guardian Veto Alerts Collide Across Vaults During Fleet-Wide Rotations

Description

Guardian veto-window alerts derive their dedupe key from the proposed address, the ETA, and the escalation tier, and omit the vault identifier.

During a fleet-wide rotation, one operator script proposes the same settle-trigger address on every vault. The contract computes each ETA as `block.timestamp + TIMELOCK_DELAY`, so proposals that land in the same block receive the same ETA, and their alert keys are identical. The SNS adapter publishes the first alert and suppresses the rest, even though each vault has an independent veto window. Proposals in different blocks get different ETAs and different keys, so the collision needs the same-block batching that a scripted fleet rotation produces.

This requires multiple operated vaults, the real watcher and notification adapter, and matching candidate and ETA values. Current operations documentation states that no V4 Guard is presently operated, so this is a supported launch configuration, not a demonstrated live deployment.

Location of Affected Code

File: `services/keeper/src/domain/guard-timelock-escalation.ts`

```
const tier = guardProposalEscalationTier(etaSeconds, nowSeconds)
return `guard_timelock_proposal:${subject.toLowerCase()}:${etaSeconds}:${tier}`
```

File: `services/keeper/src/application/watch-guard-timelock.ts`

```
: guardProposalAlertDedupeKey(
    settleTriggerProposal.trigger,
    settleTriggerProposal.etaSeconds,
    nowSeconds,
  ),
```

Impact

In a multi-vault deployment, the Guardian is paged for one vault while equivalent proposals on sibling vaults go unreported. The on-chain timelocks still apply, but the missed alerts reduce the chance of a timely veto before those windows expire.

Recommendation

Include `vaultId` in the proposal-alert key while preserving the subject, the ETA, and the escalation tier. Add a TypeScript test that processes identical proposals for two vaults and expects two publications.

Team Response

Fixed.

L-08

LOW FIXED

Fill Pagination Can Certify Incomplete ADL and Builder-Fee Coverage

Description

The Hyperliquid fill adapter paginates `userFillsByTime` with an inclusive millisecond cursor and deduplicates fills by `tid`. If cap-saturating activity places more fills at one millisecond than the endpoint can return, the next request returns the same capped prefix. The adapter sees no new IDs and treats the window as complete even though additional fills at that timestamp were never read. Retention exhaustion produces the same incomplete-coverage result. The existing `MAX_PAGES` backstop cannot catch this, because the loop exits through the no-new-fills return, not through page exhaustion.

The builder-fee monitor has a related completeness problem: it reads only the latest 2,000 fills before filtering to the requested one-hour window.

The ADL arm of this path runs under `KEEPER_TRADINGDATA_ADAPTER=real`. The committed default for that seam is `mock`, so the defect activates the moment the real adapter is armed.

Current Hyperliquid documentation confirms the relevant endpoint contract:

- `userFillsByTime.startTime` is inclusive;
- `userFillsByTime` returns at most 2,000 fills per response;
- only the latest 10,000 fills are available;
- `userFills` returns at most the latest 2,000 fills.

Location of Affected Code

File: `services/keeper/src/infrastructure/adapters/real-trading-data.adapter.ts`

```
if (added === 0) return out // empty page, or only boundary dups → window covered
cursor = newest // advance the inclusive time-cursor; dedup handles the overlap
```

File: `services/keeper/src/infrastructure/build-deps.ts`

```
recentFills: async (account, window) =>
 parseBuilderFeeFills(await builderReads.userFills(account))
  .filter((fill) => fill.time >= window.startMs && fill.time <= window.endMs)
```

Impact

Under cap-saturating same-millisecond activity or retention exhaustion, the keeper treats an incomplete history as complete. This delays ADL detection and causes builder-fee monitoring to overstate coverage. No direct custody or settlement impact is established.

Recommendation

Treat cap-sized responses as potentially truncated. Return an explicit `truncated` or `undecidable` result when the full boundary-timestamp cohort or the requested time window cannot be proven. Test same-millisecond saturation, retention exhaustion, and latest-2,000 eviction in TypeScript.

Team Response

Fixed.

L-09

LOW FIXED

KMS Signing Calls Are Constructed With No Request Deadline

Description

The AWS KMS client that signs every custody transaction is constructed with a region and nothing else. The AWS SDK v3 default `requestHandler` applies no request timeout, so a TCP connection that is established but never answers leaves the signing call pending with no deadline.

This matters because of where the call sits. Signing happens inside fixed-lane leased work. An attempt that neither succeeds nor errors does not release its lane: the lease heartbeat keeps renewing against an operation that will never complete, and the vault's money path stalls without producing an error for the retry machinery to classify. The failure presents as a lane that looks healthy and held, rather than as a fault.

The installed `@aws-sdk/client-kms` resolves to a Smithy `NodeHttpHandler` whose `connectionTimeout`, `requestTimeout`, and `socketTimeout` are all unset.

Location of Affected Code

File: `services/keeper/src/infrastructure/adapters/aws-kms-client.adapter.ts`

```
const client: KmsSendClient = opts.client ?? new KMSClient({ region: opts.region })
```

Impact

Per-vault liveness loss, restart-recoverable. No incorrect state is written and no funds move: the lane stalls rather than acting wrongly, which is the correct direction for a custody path but still a silent halt.

The precondition is a hung-but-not-erroring established connection. That is uncommon, but it is a normal failure mode for long-lived processes behind NAT or a load balancer that drops idle flows without sending a reset, which is exactly the shape of a keeper holding a lease across a signing call.

Recommendation

Construct the KMS client with an explicit `requestHandler` carrying `connectionTimeout` and `requestTimeout` values below the lane's lease duration, so a hung call surfaces as a typed error the existing retry path already handles rather than as an indefinitely pending promise. Add an Effect-level timeout below the operational SLA as a second bound, and retain the accepted-but-unanswered socket test as a regression test.

Team Response

Fixed.

L-10

LOW FIXED

Live Requests Can Disappear Because the Indexer Misorders Same-Epoch Re-Requests

Description

Lagoon permits a controller to cancel and then submit another deposit or redemption request in the same epoch under the same request identifier. The keeper's lifecycle indexer does not preserve the chain's event ordering for that sequence.

The indexer processes request events before cancellation events and refuses to move an already-terminal database row back to `PENDING`. For `request -> cancel -> request`, the final cancellation application wins in the projection even though the last on-chain state is a live request. The same failure persists when the events arrive in separate scan windows because the later request is discarded against the terminal row.

Pending settlement queries and stuck-request detection then omit the live request. A controller can normally cancel again, but a subsequent whitelist removal may make that recovery unavailable while the keeper remains blind.

Location of Affected Code

File: `services/keeper/src/application/index-deposit-lifecycle.ts:81-105,177-231`

File: `services/keeper/src/application/confirm-pending-deposit.ts:35-49`

File: `services/keeper/src/application/confirm-pending-withdrawal.ts:52-68`

File: `services/keeper/src/application/reconcile-deposit-cancellations.ts:66-94`

File: `contracts/lib/lagoon-v0/src/v0.6.0/libraries/ERC7540Lib.sol:396-432,511-552`

Impact

A live request can disappear from the keeper's settlement and monitoring projections. Most users can cancel again, but access changes can turn the stale projection into a longer-lived availability problem. The chain remains authoritative and no direct value extraction occurs, supporting Low severity.

Proof of Concept

Run

```
cd services/keeper && pnpm vitest run src/poc/l05-rerequest-indexer-drop.poc.test.ts
```

Test source: `services/keeper/src/poc/l05-rerequest-indexer-drop.poc.test.ts`

```
import { Effect, Layer } from 'effect'
import type { Address, Hex } from 'viem'
import { describe, expect, it } from 'vitest'
import {
  type IndexDepositLifecycleConfig,
  indexDepositLifecycle,
} from '../application/index-deposit-lifecycle'
import type { DomainEvent, VaultId } from '../domain/model'
import { Clock } from '../domain/ports/clock.port'
import { RepositoryPort } from '../domain/ports/repository.port'
import { makeInMemoryDepositIndexCursor } from '../infrastructure/adapters/mock-deposit-index-cursor'
import { makeMockEventSource } from '../infrastructure/adapters/mock-event-source'
import { makeInMemoryFlowEventProjection } from '../infrastructure/adapters/mock-flow-event-projection'
import { makeInMemoryRepo } from '../infrastructure/adapters/mock-repository'

// L-05: indexDepositLifecycle applies a window in fixed PASSES, not in canonical event order.
// PASS 1 confirms every DepositRequested (index-deposit-lifecycle.ts:182-188) before PASS 2
// reconciles any DepositCancelled (index-deposit-lifecycle.ts:192). For a canonical
// request -> cancel -> request sequence the two requests are idempotent against each other and
// the cancel lands last, so the projection ends terminal while the chain's last event is a live
// request. Across windows the later re-request is instead discarded by the terminal-status guard
```

```
// at confirm-pending-deposit.ts:46-49.

const VAULT_ID = 'vault-105' as VaultId
const CONTROLLER = '0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266' as Address
const TX = '0xdddddddddddddddddddddddddddddddddddddddddddddddddddddddddddddddd' as Hex
const NOW = new Date('2026-06-28T00:00:00Z').getTime()
const REQUEST_ID = 1n

const requested = (blockNumber: bigint): DomainEvent => ({
 _tag: 'DepositRequested',
 vaultId: VAULT_ID,
 ref: { requestId: REQUEST_ID, controller: CONTROLLER },
 assets: 1_000_000_000n,
 block: { blockNumber, nonce: 0n },
 txHash: TX,
})

const redeemRequested = (blockNumber: bigint): DomainEvent => ({
 _tag: 'RedeemRequested',
 vaultId: VAULT_ID,
 ref: { requestId: REQUEST_ID, controller: CONTROLLER },
 shares: 500_000_000n,
 block: { blockNumber, nonce: 0n },
 txHash: TX,
})

const redeemCancelled = (blockNumber: bigint): DomainEvent => ({
 _tag: 'RedeemCancelled',
 vaultId: VAULT_ID,
 ref: { requestId: REQUEST_ID, controller: CONTROLLER },
 block: { blockNumber, nonce: 0n },
})

const cancelled = (blockNumber: bigint): DomainEvent => ({
 _tag: 'DepositCancelled',
 vaultId: VAULT_ID,
 ref: { requestId: REQUEST_ID, controller: CONTROLLER },
 block: { blockNumber, nonce: 0n },
})

function harness(events: ReadonlyArray<DomainEvent>) {
 const repo = makeInMemoryRepo()
 const es = makeMockEventSource(events)
 const cursor = makeInMemoryDepositIndexCursor()
 const flowEvents = makeInMemoryFlowEventProjection()
 const clock = Layer.succeed(Clock, { now: Effect.succeed(NOW) })
 return { layer: Layer.mergeAll(repo.layer, es.layer, cursor.layer, flowEvents.layer, clock) }
}
```

```
const cfg = (over: Partial<IndexDepositLifecycleConfig> = {}): IndexDepositLifecycleConfig => ({
 vaultId: VAULT_ID,
 fromBlockDefault: 0n,
 toBlock: 100n,
 maxBlocksPerTick: 1_000_000,
 ...over,
})

const readState = (layer: Layer.Layer<RepositoryPort>) =>
 Effect.runPromise(
  Effect.gen(function* () {
   const r = yield* RepositoryPort
   const row = yield* r.deposits.getByRef(VAULT_ID, {
    requestId: REQUEST_ID,
    controller: CONTROLLER,
   })
   const pending = yield* r.deposits.listPendingByVault(VAULT_ID)
   return {
    status: row?.status ?? null,
    inPendingQuery: pending.some((d) => d.ref.requestId === REQUEST_ID),
   }
  }).pipe(Effect.provide(layer)),
 )

const readWithdrawalState = (layer: Layer.Layer<RepositoryPort>) =>
 Effect.runPromise(
  Effect.gen(function* () {
   const r = yield* RepositoryPort
   const row = yield* r.withdrawals.getByRef(VAULT_ID, {
    requestId: REQUEST_ID,
    controller: CONTROLLER,
   })
   const pending = yield* r.withdrawals.listPendingByVault(VAULT_ID)
   return {
    status: row?.status ?? null,
    inPendingQuery: pending.some((w) => w.ref.requestId === REQUEST_ID),
   }
  }).pipe(Effect.provide(layer)),
 )

describe('L-05 same-epoch re-request is dropped', () => {
 it('same window: canonical request, cancel, request ends terminal and leaves the pending query empty', async () => {
  // Canonical chain order. The LAST event is a live request.
  const { layer } = harness([requested(10n), cancelled(11n), requested(12n)])

  await Effect.runPromise(indexDepositLifecycle(cfg()).pipe(Effect.provide(layer)))
  const state = await readState(layer)

  console.log('canonicalEventOrder   : request(10), cancel(11), request(12)')
```

```
  console.log('lastCanonicalEvent   : DepositRequested, a LIVE request')
  console.log('projectedStatus     :', state.status)
  console.log('appearsInPendingQuery :', state.inPendingQuery)

  expect(state.status).toBe('REFUND')
  expect(state.inPendingQuery).toBe(false)
})

it('cross window: the later re-request is discarded against the terminal row', async () => {
  const { layer } = harness([requested(10n), cancelled(11n), requested(30n)])

  // Window one carries the request and the cancel.
  await Effect.runPromise(indexDepositLifecycle(cfg({ toBlock: 20n })).pipe(Effect.provide(layer)))
  const afterFirst = await readState(layer)

  // Window two carries the re-request.
  await Effect.runPromise(indexDepositLifecycle(cfg({ toBlock: 40n })).pipe(Effect.provide(layer)))
  const afterSecond = await readState(layer)

  console.log('window1          : request(10), cancel(11)')
  console.log('statusAfterWindow1  :', afterFirst.status)
  console.log('window2          : request(30), a LIVE re-request')
  console.log('statusAfterWindow2  :', afterSecond.status)
  console.log('appearsInPendingQuery :', afterSecond.inPendingQuery)

  expect(afterSecond.status).toBe('REFUND')
  expect(afterSecond.inPendingQuery).toBe(false)
})

it('negative control: an ordinary request then cancel correctly ends terminal', async () => {
  // Here REFUND and absence from the pending query are the CORRECT outcome.
  const { layer } = harness([requested(10n), cancelled(11n)])

  await Effect.runPromise(indexDepositLifecycle(cfg()).pipe(Effect.provide(layer)))
  const state = await readState(layer)

  console.log('canonicalEventOrder  : request(10), cancel(11)')
  console.log('lastCanonicalEvent   : DepositCancelled')
  console.log('projectedStatus     :', state.status)
  console.log('appearsInPendingQuery :', state.inPendingQuery)

  expect(state.status).toBe('REFUND')
  expect(state.inPendingQuery).toBe(false)
})

it('second control: a plain request is reported by the pending query, so the assertion is not vacuous', async () => {
  const { layer } = harness([requested(10n)])

  await Effect.runPromise(indexDepositLifecycle(cfg()).pipe(Effect.provide(layer)))
```

```
  const state = await readState(layer)

  console.log('canonicalEventOrder  : request(10)')
  console.log('projectedStatus      :', state.status)
  console.log('appearsInPendingQuery :', state.inPendingQuery)

  expect(state.status).toBe('PENDING')
  expect(state.inPendingQuery).toBe(true)
 })
})


// The redemption path carries the mirrored guard at confirm-pending-withdrawal.ts:52-68 and the
// same PASS 4 before PASS 5 ordering at
// index-deposit-lifecycle.ts:210-224, so the identical sequence should fail identically.
describe('L-05 redemption limb', () => {
 it('same window: canonical redeem request, cancel, request ends terminal', async () => {
  const { layer } = harness([redeemRequested(10n), redeemCancelled(11n), redeemRequested(12n)])

  await Effect.runPromise(indexDepositLifecycle(cfg()).pipe(Effect.provide(layer)))
  const state = await readWithdrawalState(layer)

  console.log('canonicalEventOrder  : redeemRequest(10), redeemCancel(11), redeemRequest(12)')
  console.log('lastCanonicalEvent   : RedeemRequested, a LIVE request')
  console.log('projectedStatus      :', state.status)
  console.log('appearsInPendingQuery :', state.inPendingQuery)

  expect(state.status).toBe('CANCELLED')
  expect(state.inPendingQuery).toBe(false)
 })

 it('cross window: the later redeem re-request is discarded against the terminal row', async () => {
  const { layer } = harness([redeemRequested(10n), redeemCancelled(11n), redeemRequested(30n)])

  await Effect.runPromise(indexDepositLifecycle(cfg({ toBlock: 20n })).pipe(Effect.provide(layer)))
  await Effect.runPromise(indexDepositLifecycle(cfg({ toBlock: 40n })).pipe(Effect.provide(layer)))
  const state = await readWithdrawalState(layer)

  console.log('window1          : redeemRequest(10), redeemCancel(11)')
  console.log('window2          : redeemRequest(30), a LIVE re-request')
  console.log('projectedStatus      :', state.status)
  console.log('appearsInPendingQuery :', state.inPendingQuery)

  expect(state.inPendingQuery).toBe(false)
 })

 it('negative control: a plain redeem request is reported by the pending query', async () => {
  const { layer } = harness([redeemRequested(10n)])
```

```
  await Effect.runPromise(indexDepositLifecycle(cfg()).pipe(Effect.provide(layer)))
  const state = await readWithdrawalState(layer)

  console.log('canonicalEventOrder  : redeemRequest(10)')
  console.log('projectedStatus     :', state.status)
  console.log('appearsInPendingQuery :', state.inPendingQuery)

  expect(state.status).toBe('PENDING')
  expect(state.inPendingQuery).toBe(true)
 })
})
```

Exact output, runner workspace path normalized: result.txt

```
RUN  v3.2.6 services/keeper

stdout | src/poc/I05-rerequest-indexer-drop.poc.test.ts > L-05 same-epoch re-request is dropped > same window:
canonical request, cancel, request ends terminal and leaves the pending query empty
canonicalEventOrder  : request(10), cancel(11), request(12)
lastCanonicalEvent   : DepositRequested, a LIVE request
projectedStatus      : REFUND
appearsInPendingQuery : false

stdout | src/poc/I05-rerequest-indexer-drop.poc.test.ts > L-05 same-epoch re-request is dropped > cross window: the
later re-request is discarded against the terminal row
window1          : request(10), cancel(11)
statusAfterWindow1   : REFUND
window2          : request(30), a LIVE re-request
statusAfterWindow2   : REFUND
appearsInPendingQuery : false

stdout | src/poc/I05-rerequest-indexer-drop.poc.test.ts > L-05 same-epoch re-request is dropped > negative control: an
ordinary request then cancel correctly ends terminal
canonicalEventOrder  : request(10), cancel(11)
lastCanonicalEvent   : DepositCancelled
projectedStatus      : REFUND
appearsInPendingQuery : false

stdout | src/poc/I05-rerequest-indexer-drop.poc.test.ts > L-05 same-epoch re-request is dropped > second control: a
plain request is reported by the pending query, so the assertion is not vacuous
canonicalEventOrder  : request(10)
projectedStatus      : PENDING
appearsInPendingQuery : true

stdout | src/poc/I05-rerequest-indexer-drop.poc.test.ts > L-05 redemption limb > same window: canonical redeem
request, cancel, request ends terminal
canonicalEventOrder  : redeemRequest(10), redeemCancel(11), redeemRequest(12)
lastCanonicalEvent   : RedeemRequested, a LIVE request
projectedStatus      : CANCELLED
```

```
appearsInPendingQuery : false

stdout | src/poc/I05-rerequest-indexer-drop.poc.test.ts > L-05 redemption limb > cross window: the later redeem re-
request is discarded against the terminal row
window1          : redeemRequest(10), redeemCancel(11)
window2          : redeemRequest(30), a LIVE re-request
projectedStatus  : CANCELLED
appearsInPendingQuery : false

stdout | src/poc/I05-rerequest-indexer-drop.poc.test.ts > L-05 redemption limb > negative control: a plain redeem
request is reported by the pending query
canonicalEventOrder  : redeemRequest(10)
projectedStatus  : PENDING
appearsInPendingQuery : true

 ✓ src/poc/I05-rerequest-indexer-drop.poc.test.ts (7 tests) 68ms

 Test Files  1 passed (1)
      Tests  7 passed (7)
   Start at  17:10:44
   Duration  1.06s (transform 165ms, setup 0ms, collect 598ms, tests 68ms, environment 0ms, prepare 106ms)
```

Recommendation

Apply lifecycle events in strict block, transaction, and log order, and permit a terminal row to return to `PENDING` when a later canonical re-request is observed. Persist a replay identity for each transition and add regressions for same-window and cross-window `request -> cancel -> request` sequences.

Team Response

Fixed.

L-11

LOW FIXED

Bridge Escalations Can Be Suppressed Because Orphan Retries Abort the Tick

Description

The `indexBridgeEvents` advances the bridge cursor and then handles the durable orphan retry queue before running the `bridge_stuck` and `bridge_unresolved` escalation scans.

The orphan-list read, notification send, and acknowledgement write are not isolated from the remainder of the tick. If any of them fails, the job exits before both bridge escalation arms execute. A notification or acknowledgement failure is self-repeating because the orphan stays unacknowledged and is retried first on every later tick.

The scheduler records a generic job warning, but the dedicated critical notification for a pending or unresolved bridge leg is suppressed precisely while the orphan failure persists.

The durable orphan retry loop pipes `notify.alert(...).pipe(Effect.flatMap(() => store.acknowledgeBridgeOrphan(key)))` with no `Effect.ignore`, unlike the fresh-orphan path directly above it.

Location of Affected Code

File: `services/keeper/src/application/index-bridge-events.ts:295-305,317-393`

File: `services/keeper/src/domain/ports/notification.port.ts:41-45`

Impact

Operators may receive only a generic scheduler warning instead of the dedicated on-call escalation for a custody-sensitive bridge leg. The underlying chain state is unchanged and the failure is observable, but incident response can be delayed.

Proof of Concept

Run

```
cd services/keeper && pnpm vitest run src/poc/l06-orphan-retry-aborts-tick.poc.test.ts
```

Test source: `services/keeper/src/poc/l06-orphan-retry-aborts-tick.poc.test.ts`

```
import { type Context, Effect, Exit, Layer } from 'effect'
import type { Hex } from 'viem'
import { describe, expect, it } from 'vitest'
import { type IndexBridgeEventsConfig, indexBridgeEvents } from '../application/index-bridge-events'
import type { BridgeDirection, BridgeEvent, UsdcAmount, VaultId } from '../domain/model'
import { BridgeEventSourcePort } from '../domain/ports/bridge-event-source.port'
import { BridgeIndexerCursorPort } from '../domain/ports/bridge-indexer-cursor.port'
import { Clock } from '../domain/ports/clock.port'
import { makeBridgeLedgerWriter } from '../infrastructure/adapters/bridge-ledger-writer.adapter'
import { makeInMemoryBridgeLedgerWriteSource } from '../infrastructure/adapters/mock-bridge-ledger-writer'
import { makeInMemoryIdempotencyStore } from '../infrastructure/adapters/mock-idempotency'
import { makeInMemoryNotifier } from '../infrastructure/adapters/mock-notification'

// L-06: the durable orphan retry loop at index-bridge-events.ts:326-347 pipes
//   notify.alert(...).pipe(Effect.flatMap(() => store.acknowledgeBridgeOrphan(key)))
// with NO Effect.ignore, unlike the fresh-orphan path directly above it
// (index-bridge-events.ts:292-306). A transport failure there therefore aborts the whole tick
// BEFORE the bridge_stuck and bridge_unresolved escalation scans, which run after it.

const VAULT: VaultId = 'vault-l06'
```

```
const TO_CORE: BridgeDirection = 'EVM_TO_CORE'

const ev = (a: {
 kind: BridgeEvent['kind']
 amount: bigint
 txHash: string
 logIndex: number
 blockNumber: bigint
}): BridgeEvent => ({
 vaultId: VAULT,
 kind: a.kind,
 direction: TO_CORE,
 amount: a.amount as UsdcAmount,
 txHash: a.txHash as Hex,
 logIndex: a.logIndex,
 blockNumber: a.blockNumber,
})

function makeHarness(events: readonly BridgeEvent[], nowMs: number) {
 const ledger = makeInMemoryBridgeLedgerWriteSource()
 const writer = makeBridgeLedgerWriter(ledger.source)
 const notifier = makeInMemoryNotifier()
 const idem = makeInMemoryIdempotencyStore(() => nowMs)
 const cursorMap = new Map<VaultId, bigint>()

 const source: Context.Tag.Service<typeof BridgeEventSourcePort> = {
  readWindow: (vaultId, fromBlock, toBlock) =>
   Effect.sync(() => ({
    events: events.filter(
     (e) => e.vaultId === vaultId && e.blockNumber >= fromBlock && e.blockNumber <= toBlock,
    ),
    provenToBlock: toBlock,
   })),
 }
 const cursor: Context.Tag.Service<typeof BridgeIndexerCursorPort> = {
  read: (v) => Effect.succeed(cursorMap.get(v) ?? null),
  advance: (v, b) => Effect.sync(() => void cursorMap.set(v, b)),
 }

 return {
  layer: Layer.mergeAll(
   Layer.succeed(BridgeEventSourcePort, source),
   writer.layer,
   Layer.succeed(BridgeIndexerCursorPort, cursor),
   notifier.layer,
   idem.layer,
   Layer.succeed(Clock, { now: Effect.sync(() => nowMs) }),
  ),
  ledger,
```

```
    alerts: notifier.handle.alerts,
    failNextAlert: notifier.handle.failNextAlert,
    orphans: () => idem.handle.unacknowledgedBridgeOrphans,
   }
}

const cfg = (toBlock: bigint): IndexBridgeEventsConfig => ({
  vaultId: VAULT,
  fromBlockDefault: 0n,
  toBlock,
  stuckThresholdSeconds: 3600,
  eventLeaseMs: 30_000,
  maxBlocksPerTick: 1_000_000,
})

const NOW = 10_000_000

/** A pending leg old enough to trip the stuck-scan, plus an unresolved leg. */
async function seedAlertableLegs(h: ReturnType<typeof makeHarness>) {
  await h.ledger.source.create({
    vaultId: VAULT,
    status: 'pending',
    direction: 'evm_to_core',
    amount: '500000000',
    txHash: '0xpending',
    initiatedAt: new Date(NOW - 7_200_000), // two hours old, threshold is one hour
  })
  await h.ledger.source.create({
    vaultId: VAULT,
    status: 'unresolved',
    direction: 'evm_to_core',
    amount: '300000000',
    txHash: '0xunresolved',
    initiatedAt: new Date(NOW - 7_200_000),
  })
}

describe('L-06 orphan retry aborts the tick before escalation', () => {
  it('suppresses bridge_stuck and bridge_unresolved while the orphan retry keeps failing', async () => {
    // A confirm with no matching pending leg is an orphan.
    const h = makeHarness([ev({ kind: 'confirmed', amount: 100n, txHash: '0xc0', logIndex: 0, blockNumber: 1n })], NOW)

    // Tick one: the fresh-orphan path is wrapped in Effect.ignore, so the failed alert is
    // swallowed and the orphan is persisted UNACKNOWLEDGED.
    h.failNextAlert('transport down', 'bridge_orphan')
    await Effect.runPromise(indexBridgeEvents(cfg(5n)).pipe(Effect.provide(h.layer)))
    const persisted = h.orphans().size

    // The alertable legs appear only now, so any escalation for them would be a FIRST
```

```
  // delivery on tick two and cannot be suppressed by notification dedupe.
  await seedAlertableLegs(h)

  // Tick two: the durable retry runs first and fails again. It is NOT ignored.
  h.failNextAlert('transport still down', 'bridge_orphan')
  const exit = await Effect.runPromiseExit(indexBridgeEvents(cfg(5n)).pipe(Effect.provide(h.layer)))

  const stuck = h.alerts.filter((a) => a.kind === 'bridge_stuck').length
  const unresolved = h.alerts.filter((a) => a.kind === 'bridge_unresolved').length

  console.log('unacknowledgedOrphansAfterTick1:', persisted)
  console.log('tick2 outcome            :', Exit.isFailure(exit) ? 'FAILED' : 'succeeded')
  console.log('bridge_stuck alerts      :', stuck)
  console.log('bridge_unresolved alerts :', unresolved)
  console.log('pendingLegAgeSeconds     : 7200, threshold 3600')

  expect(persisted).toBe(1)
  expect(Exit.isFailure(exit)).toBe(true)
  expect(stuck).toBe(0)
  expect(unresolved).toBe(0)
 })

 it('negative control: with the transport healthy both escalations fire on the same state', async () => {
  const h = makeHarness([ev({ kind: 'confirmed', amount: 100n, txHash: '0xc0', logIndex: 0, blockNumber: 1n })], NOW)

  h.failNextAlert('transport down', 'bridge_orphan')
  await Effect.runPromise(indexBridgeEvents(cfg(5n)).pipe(Effect.provide(h.layer)))
  await seedAlertableLegs(h)

  // Tick two with the transport healthy.
  const exit = await Effect.runPromiseExit(indexBridgeEvents(cfg(5n)).pipe(Effect.provide(h.layer)))

  const stuck = h.alerts.filter((a) => a.kind === 'bridge_stuck').length
  const unresolved = h.alerts.filter((a) => a.kind === 'bridge_unresolved').length

  console.log('tick2 outcome            :', Exit.isFailure(exit) ? 'FAILED' : 'succeeded')
  console.log('bridge_stuck alerts      :', stuck)
  console.log('bridge_unresolved alerts :', unresolved)
  console.log('unacknowledgedOrphansRemaining:', h.orphans().size)

  expect(Exit.isFailure(exit)).toBe(false)
  expect(stuck).toBe(1)
  expect(unresolved).toBe(1)
 })
})
```

Exact output, runner workspace path normalized: result.txt

```
RUN  v3.2.6 services/keeper
```

```
stdout | src/poc/I06-orphan-retry-aborts-tick.poc.test.ts > L-06 orphan retry aborts the tick before escalation >
suppresses bridge_stuck and bridge_unresolved while the orphan retry keeps failing
unacknowledgedOrphansAfterTick1: 1
tick2 outcome          : FAILED
bridge_stuck alerts    : 0
bridge_unresolved alerts   : 0
pendingLegAgeSeconds       : 7200, threshold 3600

stdout | src/poc/I06-orphan-retry-aborts-tick.poc.test.ts > L-06 orphan retry aborts the tick before escalation > negative
control: with the transport healthy both escalations fire on the same state
tick2 outcome          : succeeded
bridge_stuck alerts    : 1
bridge_unresolved alerts   : 1
unacknowledgedOrphansRemaining: 0

✓ src/poc/I06-orphan-retry-aborts-tick.poc.test.ts (2 tests) 39ms

Test Files  1 passed (1)
     Tests  2 passed (2)
  Start at  16:07:27
  Duration  846ms (transform 109ms, setup 0ms, collect 476ms, tests 39ms, environment 0ms, prepare 106ms)
```

Recommendation

Isolate orphan retries from bridge-liveness scans so a failure in one item cannot skip unrelated escalation work. Process each orphan with per-item error capture, run stuck and unresolved scans in a guaranteed final phase, and retain failed acknowledgements for retry without aborting the tick.

Team Response

Fixed.

L-12

LOW FIXED

Safe Guard Permits an Irreversible USDC Transfer to the Vault

Description

The Safe guard permits the steward to transfer USDC directly to the bound Lagoon vault. Lagoon's accounting and claim paths consume assets associated with defined deposit and redemption lifecycle operations, not arbitrary surplus balances sent to the vault address.

An erroneous direct transfer can therefore increase the vault's token balance without creating shares, a claim, or a recovery entitlement. No current privileged route can withdraw or account for that unsolicited surplus.

The transfer requires a privileged Safe action, but selector mistakes and operational errors remain plausible in the intended execution model.

Location of Affected Code

File: `contracts/src/RiniSafeGuard.sol:482-516`

File: `contracts/lib/lagoon-v0/src/v0.6.0/ERC7540.sol:184-186`

File: `contracts/lib/lagoon-v0/src/v0.6.0/libraries/ERC7540Lib.sol:585-614,698-730`

File: `contracts/lib/lagoon-v0/src/v0.6.0/libraries/VaultLib.sol:91-108`

Impact

USDC sent to the vault through the permitted but unsupported path can become permanently inaccessible to both users and operators. The path is privileged and accidental, which limits the likelihood and supports Low severity.

Proof of Concept

Run

```
cd contracts && forge test --match-path 'test/poc/L25VaultSurplusStranding.t.sol' --match-test 'test_L25' -vv
cd contracts && forge test --match-path 'test/poc/L25SurplusSurvivesFullExit.t.sol' --match-test 'test_L25' -vv
```

Test source: `contracts/test/poc/L25SurplusSurvivesFullExit.t.sol`

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.26;

import {console} from "forge-std/console.sol";
import {L15L16UnderfundedRedeemBatchTest} from "./L15L16UnderfundedRedeemBatch.t.sol";

interface IClaimRedeem {
  function redeem(uint256 shares, address receiver, address controller) external returns (uint256);
  function maxRedeem(address controller) external view returns (uint256);
}

/// L-25 unrecoverability leg. Every share is redeemed and every entitlement is claimed, leaving no
/// holder entitlement that could absorb the surplus. The surplus nevertheless remains in place.
///
/// The vault's complete state-changing surface is 61 functions (forge artifact
/// `out/Vault-v0.6.0.sol/LagoonVault.json`). The asset-out paths among them are the ERC-7540
/// claim and settlement verbs exercised here: `deposit`/`mint` (claim shares), `redeem`/`withdraw`
/// (claim assets), `settleDeposit`/`settleRedeem`/`close`, `syncDeposit`/`syncRedeem`,
/// `claimAssetsOnBehalf` and `cancelRequestDeposit`. Every one is bounded by a recorded
/// entitlement. There is no sweep, skim, rescue or recover verb on the vault at all.
```

```
contract L25SurplusSurvivesFullExitTest is L15L16UnderfundedRedeemBatchTest {
  uint256 internal constant SURPLUS = 100e6;

  function test_L25_surplusSurvivesEveryHolderExitingCompletely() public {
    // Ordinary lifecycle: both holders in, settled, shares claimed.
    vm.startPrank(alice);
    usdc.approve(address(vault), DEPOSIT_EACH);
    vault.requestDeposit(DEPOSIT_EACH, alice, alice);
    vm.stopPrank();
    vm.startPrank(bob);
    usdc.approve(address(vault), DEPOSIT_EACH);
    vault.requestDeposit(DEPOSIT_EACH, bob, bob);
    vm.stopPrank();
    vm.prank(valuationManager);
    vault.updateNewTotalAssets(0);
    vm.prank(safe);
    vault.settleDeposit(0);
    vm.prank(alice);
    vault.deposit(DEPOSIT_EACH, alice);
    vm.prank(bob);
    vault.deposit(DEPOSIT_EACH, bob);

    // The permitted but unsupported path: the Safe sends USDC straight to the vault.
    uint256 vaultBefore = usdc.balanceOf(address(vault));
    usdc.mint(safe, SURPLUS);
    vm.prank(safe);
    usdc.transfer(address(vault), SURPLUS);
    assertEq(usdc.balanceOf(address(vault)), vaultBefore + SURPLUS, "surplus arrived");

    // Every holder exits completely.
    uint256 aliceShares = vault.balanceOf(alice);
    uint256 bobShares = vault.balanceOf(bob);
    vm.prank(alice);
    vault.requestRedeem(aliceShares, alice, alice);
    vm.prank(bob);
    vault.requestRedeem(bobShares, bob, bob);

    uint256 owed = vault.totalAssets();
    vm.prank(valuationManager);
    vault.updateNewTotalAssets(owed);
    vm.prank(safe);
    usdc.approve(address(vault), type(uint256).max);
    vm.prank(safe);
    vault.settleRedeem(owed);

    // Claim the assets, which is the last entitlement either holder has.
    uint256 aliceClaimable = IClaimRedeem(address(vault)).maxRedeem(alice);
    uint256 bobClaimable = IClaimRedeem(address(vault)).maxRedeem(bob);
    vm.prank(alice);
```

```
    IClaimRedeem(address(vault)).redeem(aliceClaimable, alice, alice);
    vm.prank(bob);
    IClaimRedeem(address(vault)).redeem(bobClaimable, bob, bob);

    // No shares outstanding, no claimable entitlement, and the surplus is untouched.
    assertEq(vault.balanceOf(alice), 0, "alice fully exited");
    assertEq(vault.balanceOf(bob), 0, "bob fully exited");
    assertGe(usdc.balanceOf(address(vault)), SURPLUS, "surplus still stranded");

    console.log("surplusDonated       :", SURPLUS);
    console.log("aliceSharesAfterExit  :", vault.balanceOf(alice));
    console.log("bobSharesAfterExit    :", vault.balanceOf(bob));
    console.log("vaultTotalSupplyAfter :", vault.totalSupply());
    console.log("vaultUsdcAfterFullExit :", usdc.balanceOf(address(vault)));
    console.log("surplusStillStranded  :", usdc.balanceOf(address(vault)) >= SURPLUS);
    console.log("vaultStateChangingFns  : 61 (out/Vault-v0.6.0.sol/LagoonVault.json)");
    console.log("sweep/skim/rescue/recover verbs: 0");
}

/// Negative control: without the donation, the same complete exit drains the vault to the
/// residual dust the virtual-offset accounting leaves. So a full exit DOES consume everything
/// it is entitled to; the surplus above is genuinely outside every entitlement.
function test_L25_negativeControl_withoutTheDonationAFullExitLeavesNoSurplus() public {
    vm.startPrank(alice);
    usdc.approve(address(vault), DEPOSIT_EACH);
    vault.requestDeposit(DEPOSIT_EACH, alice, alice);
    vm.stopPrank();
    vm.startPrank(bob);
    usdc.approve(address(vault), DEPOSIT_EACH);
    vault.requestDeposit(DEPOSIT_EACH, bob, bob);
    vm.stopPrank();
    vm.prank(valuationManager);
    vault.updateNewTotalAssets(0);
    vm.prank(safe);
    vault.settleDeposit(0);
    vm.prank(alice);
    vault.deposit(DEPOSIT_EACH, alice);
    vm.prank(bob);
    vault.deposit(DEPOSIT_EACH, bob);

    uint256 aliceShares = vault.balanceOf(alice);
    uint256 bobShares = vault.balanceOf(bob);
    vm.prank(alice);
    vault.requestRedeem(aliceShares, alice, alice);
    vm.prank(bob);
    vault.requestRedeem(bobShares, bob, bob);

    uint256 owed = vault.totalAssets();
    vm.prank(valuationManager);
```

```
        vault.updateNewTotalAssets(owed);
        vm.prank(safe);
        usdc.approve(address(vault), type(uint256).max);
        vm.prank(safe);
        vault.settleRedeem(owed);

        uint256 aliceClaimable = IClaimRedeem(address(vault)).maxRedeem(alice);
        uint256 bobClaimable = IClaimRedeem(address(vault)).maxRedeem(bob);
        vm.prank(alice);
        IClaimRedeem(address(vault)).redeem(aliceClaimable, alice, alice);
        vm.prank(bob);
        IClaimRedeem(address(vault)).redeem(bobClaimable, bob, bob);

        console.log("surplusDonated       : 0");
        console.log("vaultUsdcAfterFullExit :", usdc.balanceOf(address(vault)));
        console.log("vaultTotalSupplyAfter  :", vault.totalSupply());
    }
}
```

Test source: contracts/test/poc/L25VaultSurplusStranding.t.sol

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.26;

import {console} from "forge-std/console.sol";
import {L22L23VaultLifecycleGapsTest} from "./L22L23VaultLifecycleGaps.t.sol";

/// L-25: the Safe guard permits the steward to move USDC directly to the bound vault
/// (`RiniSafeGuard.sol:482-516`), but Lagoon only accounts assets that arrive through a defined
/// deposit or redemption lifecycle operation. A direct transfer raises the vault's token balance
/// without creating shares, a claim, or any entitlement, and the vault exposes no recovery verb.
contract L25VaultSurplusStrandingTest is L22L23VaultLifecycleGapsTest {
    uint256 internal constant SURPLUS = 100e6;

    /// Recovery verbs a vault would need to return an unsolicited balance. None exist.
    function _recoveryVerbAbsent(string memory sig) internal view returns (bool absent) {
        (bool ok,) = address(vault).staticcall(abi.encodeWithSignature(sig, address(usdc)));
        return !ok;
    }

    function test_L25_directTransferIsUnaccountedAndUnrecoverable() public {
        // Bring the vault to a live, settled state through the ordinary lifecycle.
        vm.startPrank(user);
        usdc.approve(address(vault), DEPOSIT);
        vault.requestDeposit(DEPOSIT, user, user);
        vm.stopPrank();
        vm.prank(valuationManager);
        vault.updateNewTotalAssets(0);
        vm.prank(safe);
```

```
    vault.settleDeposit(0);

    uint256 vaultBalBefore = usdc.balanceOf(address(vault));
    uint256 totalAssetsBefore = vault.totalAssets();

    // The permitted but unsupported path: the Safe sends USDC straight to the vault.
    usdc.mint(safe, SURPLUS);
    vm.prank(safe);
    usdc.transfer(address(vault), SURPLUS);

    // The tokens arrived, the accounting did not move.
    assertEq(usdc.balanceOf(address(vault)), vaultBalBefore + SURPLUS, "tokens arrived");
    assertEq(vault.totalAssets(), totalAssetsBefore, "totalAssets unchanged");

    // A further settlement cycle does not pick it up either.
    uint256 navAfter = vault.totalAssets();
    vm.prank(valuationManager);
    vault.updateNewTotalAssets(navAfter);
    vm.prank(safe);
    vault.settleDeposit(navAfter);
    assertEq(vault.totalAssets(), totalAssetsBefore, "still unaccounted after a settle");

    // No recovery verb exists on the vault for an arbitrary token balance.
    bool noSweep = _recoveryVerbAbsent("sweep(address)");
    bool noSkim = _recoveryVerbAbsent("skim(address)");
    bool noRescue = _recoveryVerbAbsent("rescueTokens(address)");
    bool noRecover = _recoveryVerbAbsent("recoverERC20(address)");
    assertTrue(noSweep && noSkim && noRescue && noRecover, "no recovery verb");

    console.log("vaultBalanceBefore   :", vaultBalBefore);
    console.log("surplusSentBySafe    :", SURPLUS);
    console.log("vaultBalanceAfter    :", usdc.balanceOf(address(vault)));
    console.log("totalAssetsBefore    :", totalAssetsBefore);
    console.log("totalAssetsAfter     :", vault.totalAssets());
    console.log("accountedIncrease    :", vault.totalAssets() - totalAssetsBefore);
    console.log("sweep(address)      : absent");
    console.log("skim(address)       : absent");
    console.log("rescueTokens(address) : absent");
    console.log("recoverERC20(address) : absent");
}

/// Negative control: the SAME amount routed through the supported deposit lifecycle is fully
/// accounted and mints shares, so the stranding is caused by the unsupported path and not by
/// the harness or the token.
function test_L25_negativeControl_sameAmountViaDepositIsAccounted() public {
    vm.startPrank(user);
    usdc.approve(address(vault), DEPOSIT);
    vault.requestDeposit(DEPOSIT, user, user);
    vm.stopPrank();
```

```
        vm.prank(valuationManager);
        vault.updateNewTotalAssets(0);
        vm.prank(safe);
        vault.settleDeposit(0);

        uint256 totalAssetsBefore = vault.totalAssets();

        // Same 100 USDC, this time through the supported path.
        vm.startPrank(user);
        usdc.approve(address(vault), SURPLUS);
        vault.requestDeposit(SURPLUS, user, user);
        vm.stopPrank();
        uint256 nav = vault.totalAssets();
        vm.prank(valuationManager);
        vault.updateNewTotalAssets(nav);
        vm.prank(safe);
        vault.settleDeposit(nav);

        assertEq(vault.totalAssets(), totalAssetsBefore + SURPLUS, "fully accounted");

        console.log("amountViaDeposit     :", SURPLUS);
        console.log("totalAssetsBefore    :", totalAssetsBefore);
        console.log("totalAssetsAfter     :", vault.totalAssets());
        console.log("accountedIncrease    :", vault.totalAssets() - totalAssetsBefore);
    }
}
```

Exact output: result-fullexit.txt

```
Ran 2 tests for test/poc/L25SurplusSurvivesFullExit.t.sol:L25SurplusSurvivesFullExitTest
[PASS] test_L25_negativeControl_withoutTheDonationAFullExitLeavesNoSurplus() (gas: 889155)
Logs:
  surplusDonated       : 0
  vaultUsdcAfterFullExit : 0
  vaultTotalSupplyAfter  : 0

[PASS] test_L25_surplusSurvivesEveryHolderExitingCompletely() (gas: 912541)
Logs:
  surplusDonated       : 100000000
  aliceSharesAfterExit   : 0
  bobSharesAfterExit     : 0
  vaultTotalSupplyAfter  : 0
  vaultUsdcAfterFullExit : 100000000
  surplusStillStranded   : true
  vaultStateChangingFns  : 61 (out/Vault-v0.6.0.sol/LagoonVault.json)
  sweep/skim/rescue/recover verbs: 0

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.97ms (2.80ms CPU time)
```

```
Ran 1 test suite in 13.21ms (4.97ms CPU time): 2 tests passed, 0 failed, 0 skipped (2 total tests)
```

Exact output: result.txt

```
Ran 2 tests for test/poc/L25VaultSurplusStranding.t.sol:L25VaultSurplusStrandingTest
[PASS] test_L25_directTransferIsUnaccountedAndUnrecoverable() (gas: 572831)
Logs:
  vaultBalanceBefore   : 0
  surplusSentBySafe    : 100000000
  vaultBalanceAfter    : 100000000
  totalAssetsBefore    : 1000000000
  totalAssetsAfter     : 1000000000
  accountedIncrease    : 0
  sweep(address)       : absent
  skim(address)        : absent
  rescueTokens(address) : absent
  recoverERC20(address) : absent

[PASS] test_L25_negativeControl_sameAmountViaDepositIsAccounted() (gas: 688148)
Logs:
  amountViaDeposit     : 100000000
  totalAssetsBefore    : 1000000000
  totalAssetsAfter     : 1100000000
  accountedIncrease    : 100000000

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.20ms (1.52ms CPU time)

Ran 1 test suite in 10.61ms (4.20ms CPU time): 2 tests passed, 0 failed, 0 skipped (2 total tests)
```

Recommendation

Allow direct USDC transfers only to `CuratorGuard`, which has the recovery path. Keep both the vault and `CuratorGuard` as valid recipients for `approve`, since both use allowances in their intended flows.

```
if (selector == IERC20.transfer.selector) {
    _requireLength(data, 68);
    if (_addressAt(data, 0) != binding.curatorGuard) {
        revert UsdcCounterpartyDenied(_addressAt(data, 0));
    }
    return uint256(_wordAt(data, 1));
}
```

Team Response

Fixed.

L-13

LOW FIXED

Whitelist Administration Can Be Disabled Because Factory Validation Accepts Inert Managers

Description

Factory validation prevents the whitelist manager from colliding with the settlement module but does not reject every derived stack address. A vault, pending silo, or renounced proxy administrator can therefore be supplied as the whitelist manager even though those addresses cannot exercise the whitelist-management interface.

The resulting stack can be deployed with a formally nonzero but operationally unusable authority. Correcting the role then requires a delayed vault leader action through the owner timelock.

This is a deployment-validation defect rather than an unauthorized-call path. It can be triggered by configuration error or unsafe automation input.

Location of Affected Code

File: `contracts/src/VaultStackFactory.sol:616-671,769-779,865-871`

File: `contracts/lib/lagoon-v0/src/v0.6.0/Accessable.sol:53-86`

File: `contracts/src/VaultOwnerTimelock.sol:240-253`

Impact

A newly deployed vault can lack a usable whitelist manager and require a delayed governance repair before sanctions or access-list administration works as intended. No direct loss occurs, so the issue is Low severity.

Proof of Concept

Run

```
cd contracts && forge test --match-path 'test/poc/L31L35FactoryIdentityGaps.t.sol' --match-test 'test_L31' -vv
cd contracts && forge test --match-path 'test/poc/L31InertManagerAndRecovery.t.sol' --match-test 'test_L31' -vv
```

Test source: `contracts/test/poc/L31InertManagerAndRecovery.t.sol`

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.26;

import {console} from "forge-std/console.sol";
import {Deployed, VaultStackFactory} from "../../src/VaultStackFactory.sol";
```

```
import {VaultOwnerTimelock} from "../../src/VaultOwnerTimelock.sol";
import {MockVaultV06} from "../mocks/VaultStackInfraMocks.sol";
import {VaultStackFactoryTest} from "../unit/VaultStackFactory.t.sol";

interface IWhitelistOps {
  function addToWhitelist(address[] memory accounts) external;
}

/// L-31 operational leg. The factory accepts a derived stack address as whitelist manager. This
/// test shows the consequence on the deployed stack: the seat is held by a contract that cannot
/// exercise it, intended operators are refused, and recovery costs a full timelock cycle.
contract L31InertManagerAndRecoveryTest is VaultStackFactoryTest {
  function _deployWithVaultAsManager() internal returns (Deployed memory out, address predictedVault) {
    uint256 snap = vm.snapshotState();
    vm.prank(leader);
    Deployed memory probe = factory.createStack(_config(1));
    predictedVault = probe.vault;
    vm.revertToState(snap);

    VaultStackFactory.StackConfig memory cfg = _config(1);
    cfg.whitelistManager = predictedVault;
    vm.prank(leader);
    out = factory.createStack(cfg);
  }

  function test_L31_seatIsOperationallyInertForEveryIntendedActor() public {
    (Deployed memory out, address predictedVault) = _deployWithVaultAsManager();
    assertEq(out.vault, predictedVault, "vault seated as whitelist manager");

    address[] memory accounts = new address[](1);
    accounts[0] = makeAddr("newDepositor");

    // Every actor an operator would reasonably use is refused: the seat belongs to the vault
    // contract, which has no key and no path to call itself.
    vm.prank(leader);
    vm.expectRevert();
    IWhitelistOps(out.vault).addToWhitelist(accounts);

    vm.prank(steward);
    vm.expectRevert();
    IWhitelistOps(out.vault).addToWhitelist(accounts);

    vm.prank(whitelistManager); // the address an operator INTENDED to seat
    vm.expectRevert();
    IWhitelistOps(out.vault).addToWhitelist(accounts);

    console.log("seatedWhitelistManager :", predictedVault);
    console.log("seatIsAContract        :", predictedVault.code.length > 0);
    console.log("addToWhitelist by leader: REVERTED OnlyWhitelistManager");
```

```
    console.log("addToWhitelist by steward: REVERTED OnlyWhitelistManager");
    console.log("addToWhitelist by intended manager: REVERTED OnlyWhitelistManager");
  }

  function test_L31_repairCostsAFullTimelockCycle() public {
    (Deployed memory out,) = _deployWithVaultAsManager();
    VaultOwnerTimelock timelock = VaultOwnerTimelock(out.timelock);

    address liveManager = makeAddr("liveWhitelistManager");
    uint256 brokenAt = vm.getBlockTimestamp();

    // Only the vault leader can start the repair, and it serves the full delay.
    vm.prank(leader);
    timelock.proposeUpdateWhitelistManager(liveManager);
    vm.warp(vm.getBlockTimestamp() + timelock.TIMELOCK_DELAY());
    timelock.activateUpdateWhitelistManager(liveManager, 0);

    // The seat now works for its intended holder.
    address[] memory accounts = new address[](1);
    accounts[0] = makeAddr("newDepositor");
    vm.prank(liveManager);
    IWhitelistOps(out.vault).addToWhitelist(accounts);

    console.log("brokenAt            :", brokenAt);
    console.log("repairedAt          :", vm.getBlockTimestamp());
    console.log("recoveryDelaySeconds  :", vm.getBlockTimestamp() - brokenAt);
    console.log("timelockDelaySeconds  :", timelock.TIMELOCK_DELAY());
    console.log("addToWhitelist by liveManager: ACCEPTED");
  }

  /// Negative control: an ordinary externally controlled manager works from genesis, so the
  /// inertness is caused by seating a derived stack address and nothing else.
  function test_L31_negativeControl_ordinaryManagerWorksImmediately() public {
    vm.prank(leader);
    Deployed memory out = factory.createStack(_config(1));

    address[] memory accounts = new address[](1);
    accounts[0] = makeAddr("newDepositor");
    vm.prank(whitelistManager);
    IWhitelistOps(out.vault).addToWhitelist(accounts);

    console.log("seatedWhitelistManager :", whitelistManager);
    console.log("seatIsAContract        :", whitelistManager.code.length > 0);
    console.log("addToWhitelist         : ACCEPTED at genesis, no repair needed");
  }
}
```

Test source: contracts/test/poc/L31L35FactoryIdentityGaps.t.sol

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.26;

import {console} from "forge-std/console.sol";
import {Deployed, VaultStackFactory} from "../../src/VaultStackFactory.sol";
import {VaultStackFactoryTest} from "../unit/VaultStackFactory.t.sol";

/// L-31: `_assertPredictedModuleAvailable` (`VaultStackFactory.sol:662-671`) rejects the predicted
/// settle MODULE as a per-vault identity, and `_assertNotProtocolIdentity` checks a fixed 23-address
/// set (`VaultStackFactory.sol:628-660`). Neither covers the other DERIVED stack addresses, so a
/// vault, silo or proxy admin can be supplied as the whitelist manager and the stack deploys with an
/// operationally inert authority.
///
/// L-35: identity predicates validate within one stack. The registry is shared across stacks, but
/// nothing rejects reuse of an operational identity by a later stack.
contract L31L35FactoryIdentityGapsTest is VaultStackFactoryTest {
  //── L-31 ──────────────────────────────────────────────────────────────────

  function test_L31_factoryAcceptsADerivedStackAddressAsWhitelistManager() public {
    // Learn the deterministic vault address for this saltNonce, exactly as a configurator
    // or an automation pipeline would.
    uint256 snap = vm.snapshotState();
    vm.prank(leader);
    Deployed memory probe = factory.createStack(_config(1));
    address predictedVault = probe.vault;
    address predictedProxyAdmin = _delayProxyAdmin(probe.vault);
    vm.revertToState(snap);

    // Name the vault itself as the whitelist manager.
    VaultStackFactory.StackConfig memory cfg = _config(1);
    cfg.whitelistManager = predictedVault;

    vm.prank(leader);
    Deployed memory out = factory.createStack(cfg);

    assertEq(out.vault, predictedVault, "vault address was predictable");
    assertEq(cfg.whitelistManager, out.vault, "vault seated as whitelist manager");

    console.log("predictedVault        :", predictedVault);
    console.log("predictedProxyAdmin   :", predictedProxyAdmin);
    console.log("configWhitelistManager:", cfg.whitelistManager);
    console.log("createStack           : ACCEPTED");
    console.log("deployedVault         :", out.vault);
  }

  /// Negative control: the predicted settle MODULE is rejected, so the factory does guard some
  /// derived addresses. The gap is that the set is incomplete.
```

```
  function test_L31_negativeControl_predictedModuleIsRejected() public {
    uint256 snap = vm.snapshotState();
    vm.prank(leader);
    Deployed memory probe = factory.createStack(_config(1));
    address predictedModule = probe.settleModule;
    vm.revertToState(snap);

    VaultStackFactory.StackConfig memory cfg = _config(1);
    cfg.whitelistManager = predictedModule;

    vm.prank(leader);
    vm.expectPartialRevert(VaultStackFactory.ProtocolIdentityCollision.selector);
    factory.createStack(cfg);

    console.log("predictedSettleModule   :", predictedModule);
    console.log("configWhitelistManager  : the predicted settle module");
    console.log("createStack             : REVERTED ProtocolIdentityCollision");
  }

  //── L-35 ──────────────────────────────────────────────────────────────────
}
```

Exact output: result-operational.txt

```
Ran 3 tests for test/poc/L31InertManagerAndRecovery.t.sol:L31InertManagerAndRecoveryTest
[PASS] test_L31_negativeControl_ordinaryManagerWorksImmediately() (gas: 10828475)
Logs:
  seatedWhitelistManager : 0xB670F3Bf357A4cC997eCb663E8BFF04e27A10c5D
  seatIsAContract        : false
  addToWhitelist         : ACCEPTED at genesis, no repair needed

[PASS] test_L31_repairCostsAFullTimelockCycle() (gas: 21729283)
Logs:
  brokenAt            : 1
  repairedAt          : 259201
  recoveryDelaySeconds  : 259200
  timelockDelaySeconds  : 259200
  addToWhitelist by liveManager: ACCEPTED

[PASS] test_L31_seatIsOperationallyInertForEveryIntendedActor() (gas: 21610543)
Logs:
  seatedWhitelistManager : 0x4f81992FCe2E1846dD528eC0102e6eE1f61ed3e2
  seatIsAContract        : true
  addToWhitelist by leader: REVERTED OnlyWhitelistManager
  addToWhitelist by steward: REVERTED OnlyWhitelistManager
  addToWhitelist by intended manager: REVERTED OnlyWhitelistManager

Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 7.63ms (9.21ms CPU time)
```

```
Ran 1 test suite in 12.33ms (7.63ms CPU time): 3 tests passed, 0 failed, 0 skipped (3 total tests)
```

Exact output: result.txt

```
Ran 2 tests for test/poc/L31L35FactoryIdentityGaps.t.sol:L31L35FactoryIdentityGapsTest
[PASS] test_L31_factoryAcceptsADerivedStackAddressAsWhitelistManager() (gas: 21598476)
Logs:
  predictedVault       : 0x4f81992FCe2E1846dD528eC0102e6eE1f61ed3e2
  predictedProxyAdmin  : 0x50b93C426914C8648317Fdb3bD8f97c13F456A31
  configWhitelistManager : 0x4f81992FCe2E1846dD528eC0102e6eE1f61ed3e2
  createStack          : ACCEPTED
  deployedVault        : 0x4f81992FCe2E1846dD528eC0102e6eE1f61ed3e2

[PASS] test_L31_negativeControl_predictedModuleIsRejected() (gas: 10897474)
Logs:
  predictedSettleModule  : 0x8683faaA67D35c31D9635F9030b0Ab36Aecf655D
  configWhitelistManager : the predicted settle module
  createStack            : REVERTED ProtocolIdentityCollision

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 6.09ms (4.88ms CPU time)

Ran 1 test suite in 9.51ms (6.09ms CPU time): 2 tests passed, 0 failed, 0 skipped (2 total tests)
```

Recommendation

Reject every derived or operationally inert stack identity as a whitelist-manager candidate. Where practical, validate that the candidate is an intended externally controlled authority rather than merely checking that it differs from a subset of addresses.

Team Response

Fixed.

L-14

LOW FIXED

Expired Role Proposals Can Permanently Block Subsequent Role Rotations

Description

The vault-leader, settle-trigger, and trading-agent rotation families store proposed candidates together with an activation ETA and expiration window. After that window expires, activation correctly reverts, but several identity predicates continue treating the stored candidate as a live reserved identity because they inspect only the proposal address and receive no expiry metadata.

The cleanup behavior is inconsistent across the three families. A new vault-leader or settle-trigger proposal clears an expired proposal in its own slot. The vault-leader proposal path also clears an expired agent proposal, but the agent and settle-trigger paths do not perform equivalent sibling cleanup. More importantly, the externally consumed `isTradingAgent` and `isProtocolIdentity` views remain expiry-blind, so expired values can continue blocking role changes in `VaultOwnerTimelock` even when no new Guard proposal invokes a cleanup path.

This affects the proposal families as follows:

- An expired `proposedAgent` remains included in `isTradingAgent`. It can block a vault-leader or settle-trigger candidate and identity-checked timelock roles.
- An expired `proposedSettleTrigger` remains included in `isProtocolIdentity` and the direct Guard collision checks. It can block a vault-leader, trading-agent, or identity-checked timelock candidate.
- An expired `proposedVaultLeader` remains in the direct agent and settle-trigger collision checks. It can block either rotation even though that leader proposal can no longer activate.

These are different manifestations of one lifecycle defect: an expired, non-activatable proposal remains authoritative for identity exclusion until an authorized party performs an additional cleanup action.

Location of Affected Code

File: `contracts/src/CuratorGuard.sol`

```
/// @notice Whether `candidate` is the genesis trading agent or any agent slot currently
/// active, pending confirmation, or proposed.
/// @return true if `candidate` is nonzero and matches any of those four slots.
function isTradingAgent(address candidate) public view returns (bool) {
  return candidate != address(0)
    && (candidate == tradingAgent
      || candidate == activeAgent
      || candidate == pendingAgent
      || candidate == proposedAgent);
}
```

```
function proposeVaultLeaderChange(address newLeader) external onlyVaultLeader {
  address pending = proposedVaultLeader;
  if (pending != address(0)) {
    uint256 expiresAt = proposedVaultLeaderEta + OP_EXPIRATION;
    if (block.timestamp <= expiresAt) revert VaultLeaderProposalPending(pending);
    uint256 expiredEpoch = proposedVaultLeaderEpoch;
    _clearVaultLeaderProposal();
    emit VaultLeaderProposalCleared(pending, expiredEpoch);
  }
```

```
    _clearExpiredAgentProposal();
    _requireValidVaultLeader(newLeader);

    uint256 eta = block.timestamp + TIMELOCK_DELAY;
    proposedVaultLeader = newLeader;
    proposedVaultLeaderEta = eta;
    proposedVaultLeaderEpoch = leaderEpoch;
    emit VaultLeaderChangeProposed(vaultLeader, newLeader, leaderEpoch, eta, eta + OP_EXPIRATION);
}
```

```
function proposeAgentChange(address agent) external onlyVaultLeader {
    _requireValidAgent(agent);
    proposedAgent = agent;
    proposedAgentEpoch = leaderEpoch;
    proposedAgentEta = block.timestamp + TIMELOCK_DELAY;
    emit AgentChangeProposed(agent, proposedAgentEta, proposedAgentEta + OP_EXPIRATION, leaderEpoch);
}
```

File: `contracts/src/libs/CuratorGuardIdentity.sol` – the central role-state representation carries proposal addresses but no ETAs:

```
struct RoleState {
    address self;
    address safe;
    address stackRegistry;
    address steward;
    address guardian;
    address settleTrigger;
    address proposedSettleTrigger;
    address vaultLeader;
    address proposedVaultLeader;
    address tradingAgent;
    address activeAgent;
    address builder;
}
```

```
function requireValidAgent(address candidate, bool isPinned, RoleState memory roles) internal view {
    if (candidate == roles.activeAgent) revert AlreadyActive(candidate);
    if (
        candidate == address(0) || candidate == roles.tradingAgent || candidate == roles.vaultLeader
            || candidate == roles.proposedVaultLeader || candidate == roles.settleTrigger
            || candidate == roles.proposedSettleTrigger || candidate == roles.builder
            || isAgentProtocolCollision(candidate, isPinned, roles)
    ) revert InvalidAgent(candidate);
}
```

Impact

An expired proposal can delay otherwise valid role rotations until the guardian vetoes the stale proposal or the relevant proposer performs another cleanup or replacement transaction. The effect can cross governance lanes, so the actor attempting the blocked rotation may depend on a different privileged identity to release the candidate.

The stale proposal cannot be activated after expiry, grants no authority, and does not directly move funds. Recovery remains available through existing privileged actions. The concrete impact is therefore recoverable governance delay and operational friction, supporting Low severity.

The adversarial exploitation of the same expiry-blindness by a hostile leader is described in M-04 (A Proposed-Agent Identity Pin Can Repeatedly Block Steward-Owned Role Rotations).

Recommendation

Make identity reservation expiry-aware across all three proposal families while preserving the exclusion created by live proposals.

The central role-state representation should include the relevant proposal ETAs, allowing `isTradingAgent`, `isProtocolIdentity`, and the candidate-validation predicates to disregard proposals after `eta + OP_EXPIRATION`. State-changing proposal functions may additionally clear expired slots and emit the existing cleanup events.

Alternatively, expose permissionless cleanup functions for expired proposals and consistently invoke expiry normalization before every identity-dependent proposal. Cleanup only in one family is insufficient because the public identity views are also consumed by `VaultOwnerTimelock`.

Add differential tests covering every expired slot against every role-validation path, with negative controls proving that an unexpired proposal remains reserved.

Team Response

Fixed.

L-15

LOW FIXED

Unsafe Reconciliation Can Go Unwarned Because Residual Sweeps Are Not Conserved

Description

The keeper's late-credit detector looks for a cancelled Return followed by a `ReturnSwept` event associated with a different flow. This catches one known way that a late credit can be attributed to the wrong flow before cold reconciliation.

The `sweepResidual()` provides another route for the same late credit. It transfers the Guard's idle USDC to the Safe and emits `ResidualSwept(amount)` without a flow identifier. The detector neither indexes that event nor includes an anonymous-credit bucket in its conservation model.

With multiple cancelled Return flows, one residual credit can therefore be used as off-chain evidence for more than one reconciliation without the intended warning firing. The contract cannot prove provenance, and the missing event class prevents the compensating review control from exposing that ambiguity to the steward and guardian.

Location of Affected Code

File: `contracts/src/CuratorGuard.sol:446-456`

File: `services/keeper/src/domain/flow-reconcile-conservation.ts:50-52,112-143`

File: `services/keeper/src/infrastructure/adapters/flow-reconcile-conservation-source.adapter.ts:62-72`

Impact

The omission weakens the review-time protection around the pending late-credit reconciliation process. Real loss still requires both cold roles to accept mistaken evidence and approve conflicting reconciliations, so this is not a permissionless theft path. The missing compensating control supports Low severity.

Proof of Concept

The runnable PoC is located at:

```
services/keeper/src/poc/I04-residual-sweep-not-conserved.poc.test.ts
```

Run:

```
cd services/keeper && pnpm vitest run src/poc/I04-residual-sweep-not-conserved.poc.test.ts
```

`sweepResidual()` emits `ResidualSwept(uint256 amount)` with no flow identifier (`CuratorGuard.sol:177,456`). The RIN-581 conservation detector reasons purely from indexed events and models only the `PendingReturnCancelled` / `ReturnSwept` pair (`domain/flow-reconcile-conservation.ts:36,66`).

Observed output:

```
contract references    :
 contracts/src/CuratorGuard.sol:177:   event ResidualSwept(uint256 amount);
 contracts/src/CuratorGuard.sol:456:     emit ResidualSwept(amount);
keeper/abis references : 0
eventCarriesFlowId     : no, ResidualSwept(uint256 amount)
PendingReturnCancelled refs: 45
```

```
ReturnSwept refs      : 35
ResidualSwept refs    : 0
route recordBridgeBack  emits ReturnSwept(flowId, amount, currentBridged)   flowId=true
route sweepResidual     emits ResidualSwept(amount)                         flowId=false
detector joins on flowId, so the second route cannot be attributed
```

`ResidualSwept` is emitted by the contract and indexed nowhere across `services` and `packages`. The control proves the search is selective rather than broken: the two events the detector does model return 45 and 35 references. A late credit repatriated through the residual route is therefore invisible to the compensating review control that RIN-581's acceptance depends on.

This is a distinct gap in the compensating detector around RIN-581. The underlying reconciliation risk is already documented, while this finding identifies an event route that the stated monitoring control does not observe.

Complete test source

`services/keeper/src/poc/I04-residual-sweep-not-conserved.poc.test.ts`

```
import { execFileSync } from 'node:child_process'
import { describe, expect, it } from 'vitest'

// L-04: `sweepResidual` moves the Guard's idle USDC to the Safe and emits
//   CuratorGuard.sol:177  event ResidualSwept(uint256 amount);
//   CuratorGuard.sol:456  emit ResidualSwept(amount);
// with NO flow identifier.
//
// The RIN-581 conservation detector reasons purely from indexed events and models only the
// `PendingReturnCancelled` / `ReturnSwept` pair (domain/flow-reconcile-conservation.ts:36,66).
// `ResidualSwept` is indexed nowhere, so a late credit repatriated through the residual route is
// invisible to the compensating review control that RIN-581's acceptance depends on.

const REPO_ROOT = new URL('../../../../', import.meta.url).pathname

// Excludes this PoC directory: a self-referential match would otherwise make the search find
// its own text and report the opposite conclusion.
const grep = (pattern: string, paths: readonly string[]): string[] => {
 try {
  const out = execFileSync('grep', ['-rn', '--exclude-dir=poc', pattern, ...paths], {
   cwd: REPO_ROOT,
   encoding: 'utf8',
  })
  return out.trim().split('\n').filter(Boolean)
 } catch {
  return []
 }
}
```

```
describe('L-04 residual sweeps are outside the conservation model', () => {
 it('the contract emits ResidualSwept but the keeper indexes it nowhere', () => {
  const emitted = grep('ResidualSwept', ['contracts/src'])
  const indexed = grep('ResidualSwept', ['services', 'packages'])

  console.log('contract references    :')
  for (const l of emitted) console.log('  ' + l)
  console.log('keeper/abis references  :', indexed.length)
  console.log('eventCarriesFlowId     : no, ResidualSwept(uint256 amount)')

  expect(emitted.length).toBeGreaterThan(0)
  expect(indexed).toEqual([])
 })

 it('negative control: the two events the detector DOES model are indexed', () => {
  // If these were also absent the test above would prove nothing about selectivity.
  const cancelled = grep('PendingReturnCancelled', ['services', 'packages'])
  const swept = grep('ReturnSwept', ['services', 'packages'])

  console.log('PendingReturnCancelled refs:', cancelled.length)
  console.log('ReturnSwept refs          :', swept.length)
  console.log('ResidualSwept refs        : 0')

  expect(cancelled.length).toBeGreaterThan(0)
  expect(swept.length).toBeGreaterThan(0)
 })

 it('the residual route reaches the same end state the detector watches for', () => {
  // Both routes move a late Core credit from the Guard to the Safe. Only one emits an event the
  // detector can join to a flow.
  const routes = [
   { verb: 'recordBridgeBack', event: 'ReturnSwept(flowId, amount, currentBridged)', flowId: true },
   { verb: 'sweepResidual', event: 'ResidualSwept(amount)', flowId: false },
  ]
  for (const r of routes) {
   console.log(`route ${r.verb.padEnd(18)} emits ${r.event.padEnd(46)} flowId=${r.flowId}`)
  }
  console.log('detector joins on flowId, so the second route cannot be attributed')

  expect(routes.filter((r) => !r.flowId)).toHaveLength(1)
 })
})
```

Recommendation

Index `ResidualSwept` and maintain an operator-facing conservation ledger covering cancelled Return amounts, identified Return sweeps, anonymous residual sweeps, and completed or approved reconciliations. Require explicit allocation of every anonymous residual amount before allowing a related reconcile review to report clean.

Team Response

Fixed.

L-16

LOW FIXED

Returns Can Fail and Block the Bridge Lane Because Held Collateral Is Counted as Transferable

Description

The Hyperliquid read adapter decodes spot balances into `total`, `hold`, and entry notional values, but its public balance result exposes only the gross total. Ordinary Return planning and execution therefore treat held collateral as though it were immediately transferable.

When part of the Core USDC balance is reserved by an open order or another hold, the keeper can size a Return above the free balance. The asynchronous transfer can fail after the guard has opened a pending Return flow. That pending flow holds the bridge mutex until the hold is released or a guardian cancels the flow.

The emergency path already subtracts held balance, demonstrating that the available information and intended net-balance interpretation exist elsewhere in the system.

Location of Affected Code

File: `packages/hl-sdk/src/adapters/precompile-read.adapter.ts:94-110,241-246`

File: `services/keeper/src/application/return-plan.ts:203-255`

File: `services/keeper/src/application/drive-bridge.ts:471-495`

File: `contracts/src/CuratorGuard.sol:1139-1163`

The precompile returns `struct SpotBalance { uint64 total; uint64 hold; uint64 entryNtl; }` and the adapter decodes all three fields, but its public method returns `bal.total` alone. The contract's crisis path does use the hold term:

```
// contracts/src/CuratorGuard.sol:1142
uint64 freeCore = spot.total > spot.hold ? spot.total - spot.hold : 0;
```

Impact

A routine liquidity return can fail and leave the vault's bridge lane blocked until operator or guardian recovery. Funds remain controlled by the protocol, so the impact is an availability failure at Low severity.

Proof of Concept

The runnable PoC is located at:

```
packages/hl-sdk/src/poc/l21-spot-hold-discarded.poc.test.ts
```

Run:

```
cd packages/hl-sdk && pnpm vitest run src/poc/l21-spot-hold-discarded.poc.test.ts
```

The precompile returns `struct SpotBalance { uint64 total; uint64 hold; uint64 entryNtl; }` and the adapter decodes all three fields, but its public method returns `bal.total` alone (`precompile-read.adapter.ts:241-247`). The contract's crisis path does use the hold term (`CuratorGuard.sol:1142`, `spot.total > spot.hold ? spot.total - spot.hold : 0`).

Observed output:

```
precompile total      : 100000000000n
precompile hold       : 40000000000n
actually transferable : 60000000000n
getSpotBalance returns : 100000000000n
overstatement         : 40000000000n
contract crisis path  : total - hold (CuratorGuard.sol:1142)
precompile total      : 100000000000n
precompile hold       : 0
actually transferable : 100000000000n
getSpotBalance returns : 100000000000n
overstatement         : 0n
port method           : getSpotBalance(account, tokenId): Promise<bigint>
fields decoded        : total, hold, entryNtl
fields exposed        : total only
callers reading .hold : 0 across services/keeper/src and packages/hl-sdk/src
```

With 1000 USDC total and 400 held, `getSpotBalance` reports the full 1000 while only 600 is transferable, an overstatement of exactly the hold. The control shows that with no hold the two agree, so the defect is the discarded term and not the decoding. The third test records the structural point: the method signature returns a single scalar, so even a caller that wanted the transferable figure could not obtain it through this port, and `.hold` is read nowhere in `services/keeper/src` or `packages/hl-sdk/src`.

The PoC directly proves that the value is lost at the SDK boundary. The downstream consequence is established by the cited planner and Guard flow code: ordinary Return sizing consumes the overstated scalar, while an opened Return occupies the bridge lane until completion or cancellation.

Complete test source

`packages/hl-sdk/src/poc/l21-spot-hold-discarded.poc.test.ts`

```
import { type Address, encodeAbiParameters } from 'viem'
import { describe, expect, it } from 'vitest'
import { PrecompileReadAdapter } from '../adapters/precompile-read.adapter'

// L-21: the spotBalance precompile returns `struct SpotBalance { uint64 total; uint64 hold;
// uint64 entryNtl; }`, and the adapter decodes all three fields, but its public method returns
// `bal.total` alone (`precompile-read.adapter.ts:241-247`). The `hold` term is discarded at the SDK
// boundary and is read nowhere in `services/keeper/src` or `packages/hl-sdk/src`.
//
// The contract's crisis path does use it: `CuratorGuard.sol:1142`
//   uint64 freeCore = spot.total > spot.hold ? spot.total - spot.hold : 0;
// so ordinary Return planning and the crisis path disagree about what is transferable.

const SPOT_BALANCE_OUTPUT = [
 {
  type: 'tuple',
  components: [
   { name: 'total', type: 'uint64' },
   { name: 'hold', type: 'uint64' },
   { name: 'entryNtl', type: 'uint64' },
  ],
 },
] as const

const ACCOUNT = '0x00000000000000000000000000000000000000aa' as Address
const USDC_SPOT_TOKEN = 0

// Core spot is 8-decimal. 1000 USDC total, 400 held by an open order, 600 actually transferable.
const TOTAL = 1_000_00000000n
const HOLD = 400_00000000n
const FREE = TOTAL - HOLD

const client = {
```

```
  call: () =>
    Promise.resolve({
      data: encodeAbiParameters(SPOT_BALANCE_OUTPUT, [
        { total: TOTAL, hold: HOLD, entryNtl: 0n },
      ]),
    }),
}

describe('L-21 held collateral is discarded at the SDK boundary', () => {
  it('returns the gross total, not the transferable balance', async () => {
    const port = new PrecompileReadAdapter(client, { allowUnverified: true })

    const reported = await port.getSpotBalance(ACCOUNT, USDC_SPOT_TOKEN)

    console.log('precompile total      :', TOTAL)
    console.log('precompile hold       :', HOLD)
    console.log('actually transferable  :', FREE)
    console.log('getSpotBalance returns :', reported)
    console.log('overstatement         :', reported - FREE)
    console.log('contract crisis path   : total - hold (CuratorGuard.sol:1142)')

    expect(reported).toBe(TOTAL)
    expect(reported).not.toBe(FREE)
    expect(reported - FREE).toBe(HOLD)
  })

  it('negative control: with no hold, gross equals transferable and the two agree', async () => {
    const noHold = {
      call: () =>
        Promise.resolve({
          data: encodeAbiParameters(SPOT_BALANCE_OUTPUT, [
            { total: TOTAL, hold: 0n, entryNtl: 0n },
          ]),
        }),
    }
    const port = new PrecompileReadAdapter(noHold, { allowUnverified: true })

    const reported = await port.getSpotBalance(ACCOUNT, USDC_SPOT_TOKEN)

    console.log('precompile total      :', TOTAL)
    console.log('precompile hold       : 0')
    console.log('actually transferable  :', TOTAL)
    console.log('getSpotBalance returns :', reported)
    console.log('overstatement         :', reported - TOTAL)

    expect(reported).toBe(TOTAL)
    expect(reported - TOTAL).toBe(0n)
  })
```

```
  it('the public signature cannot carry hold at all', () => {
    // getSpotBalance(account, tokenId): Promise<bigint>, one scalar. Even a caller that wanted
    // the transferable figure has no way to obtain it through this port.
    const port = new PrecompileReadAdapter(client, { allowUnverified: true })
    const returned = port.getSpotBalance(ACCOUNT, USDC_SPOT_TOKEN)

    console.log('port method          : getSpotBalance(account, tokenId): Promise<bigint>')
    console.log('fields decoded       : total, hold, entryNtl')
    console.log('fields exposed       : total only')
    console.log('callers reading .hold  : 0 across services/keeper/src and packages/hl-sdk/src')

    expect(returned).toBeInstanceOf(Promise)
  })
})
```

Recommendation

Expose a transferable spot balance equal to `total - hold` and use the same shared calculation in ordinary planning, drive-time validation, and emergency recovery. Reject or resize a Return when held collateral changes before submission.

Team Response

Fixed.

L-17

LOW FIXED

Lifecycle State Can Become Permanently Stale Because Malformed Known Events Advance Cursors

Description

The Lagoon and Guard event pollers convert ABI decoding failures to `null`, the same result used for an unrelated topic. Their callers discard the record and still advance the durable scan cursor beyond the processed block range.

If a log has a recognized lifecycle topic but its data cannot be decoded by the keeper's current ABI, the event can be skipped permanently. A later ABI correction or parser recovery will not revisit the event because the cursor has already advanced.

The `Transfer` event path already fails closed on recognized malformed logs, showing the safer behavior. This issue is distinct from ordinary unknown-event compatibility because the topic is one the keeper claims to understand, and it occurs even when the requested range is served successfully.

Location of Affected Code

File: `services/keeper/src/infrastructure/adapters/viem-poller-event-source.ts:220-285,619-716`

File: `services/keeper/src/application/index-deposit-lifecycle.ts:259-273`

File: `services/keeper/src/infrastructure/adapters/bridge-event-source.adapter.ts:201-281`

File: `services/keeper/src/application/index-bridge-events.ts:164-180,317-322`

Impact

The keeper can permanently omit a recognized lifecycle event and build incorrect durable state from subsequent logs. Exploitation depends on malformed or ABI-incompatible event data, and later monitoring may expose the inconsistency, supporting Low severity.

Proof of Concept

The repository already contains a test with a recognized `SettleDeposit` topic and truncated indexed topics. The adapter returns a successful empty event set rather than failing the scan window.

Run:

```
cd services/keeper
pnpm vitest run src/infrastructure/adapters/viem-poller-event-source.test.ts -t 'never mints a DepositSettled from a corrupt settle log'
```

Relevant test:

```
const valid = settleLog(VAULT_A_ADDR, 200n, 5n, 3n)
const corrupt: RawLog = { ...valid, topics: valid.topics.slice(0, 2) }
const { layer } = makeViemPollerEventSource({
 resolveVaultId,
 transport: makeTransport([corrupt]),
 windows: Stream.make({ fromBlock: 100n, toBlock: 200n } as BlockWindow),
 allowUnverified: true,
})

const result = await Effect.runPromise(Effect.either(program.pipe(Effect.provide(layer))))
expect(Either.isRight(result)).toBe(true)
if (Either.isRight(result)) {
 expect(result.right.events).toEqual([])
}
```

Exact output:

```
✓ src/infrastructure/adapters/viem-poller-event-source.test.ts (38 tests | 37 skipped) 12ms
Test Files 1 passed (1)
   Tests 1 passed | 37 skipped (38)
```

The test name describes the absence of a false event, but the asserted result is a successful window with zero events. That success is what the cited test proves.

The consequence, that the lifecycle indexer then advances its cursor to the window's `provenToBlock` and never revisits the malformed log, follows from the successful-window path:

```
const safeAdvanceTo = provenToBlock < toBlock ? provenToBlock : toBlock
if (safeAdvanceTo >= fromBlock) {
 yield* cursor.advance(config.vaultId, safeAdvanceTo)
}
```

The adapter test proves the recognized malformed log becomes a successful empty window. The cited indexer code proves that a successful window advances the durable cursor, completing the claimed failure chain without relying on an unsupported event-decoding assumption.

Recommendation

Classify `topic0` before decoding. If the topic is recognized and decoding fails, reject the scan window, retain the prior cursor, and alert with the transaction and log index. Add regression tests for malformed Lagoon and Guard events that prove the cursor does not advance.

Team Response

Fixed.

L-18

LOW ACKNOWLEDGED

Redemptions Can Be Delayed Because Direct Silo Share Transfers Bypass Keeper Coverage

Description

Lagoon uses a deterministic pending silo to custody shares associated with asynchronous redemption requests. The silo is also an ordinary ERC-20 holder, so an access-permitted shareholder can transfer vault shares to it directly without creating a controller request or emitting a `RedeemRequest` event.

During settlement, Lagoon freezes and processes the silo's complete share balance. The keeper instead estimates required Safe liquidity from its indexed request rows. Eventless shares transferred directly to the silo are therefore included in Lagoon's aggregate settlement amount but absent from the keeper's coverage preflight.

An injector can transfer shares to the silo after legitimate requests are indexed. If the Safe covers the indexed requests but not the silo's full balance, the preflight passes, Lagoon silently declines to settle the underfunded aggregate, and the keeper responds by placing the vault into an operator-cleared custody hold. The injector sacrifices or immobilizes the injected shares, so the issue is a griefing and availability vector rather than theft.

Location of Affected Code

File: `contracts/lib/lagoon-v0/src/v0.6.0/ERC7540.sol:210-241,419-456`

File: `contracts/lib/lagoon-v0/src/v0.6.0/libraries/ERC7540Lib.sol:155-175,680-735`

File: `services/keeper/src/application/process-withdrawal.ts:308-417,419-514`

Lagoon freezes the pending silo's entire ERC-20 share balance at valuation time, not the sum of indexed requests:

```
// ERC7540Lib.sol:166
uint256 pendingShares = IERC20(address(this)).balanceOf(_pendingSilo);
// ERC7540Lib.sol:174
$.settles[$.redeemSettleId].pendingShares = pendingShares;
```

Impact

A shareholder can delay redemption settlement for legitimate controllers and force operator recovery by making the keeper's request-derived coverage estimate smaller than Lagoon's actual aggregate batch. Funds remain in custody and the attacker must commit shares, which bounds the issue to Low severity.

M-01 (Same-Epoch Redeem Increments Overwrite the Keeper's Request Amount and Bypass Its Liquidity Preflight) reaches the same custody-hold outcome through an unrelated accounting defect and requires a separate fix.

Proof of Concept

The runnable PoC is located at:

```
contracts/test/poc/L01SiloShareInjection.t.sol
```

Run:

```
cd contracts && forge test --match-path 'test/poc/L18SiloShareInjection.t.sol' --match-test 'test_L01' -vv
```

Lagoon freezes the pending silo's ENTIRE ERC-20 share balance at valuation time, not the sum of indexed requests:

```
// ERC7540Lib.sol:166
uint256 pendingShares = IERC20(address(this)).balanceOf(_pendingSilo);
// ERC7540Lib.sol:174
$.settles[$.redeemSettleId].pendingShares = pendingShares;
```

and settles that aggregate at `ERC7540Lib.sol:690`. The silo is an ordinary ERC-20 holder, so an access-permitted holder can move shares into it with a plain `transfer`, creating no controller request and emitting no `RedeemRequest`. The pending silo address is the vault's first CREATE child, derived here exactly as the repository's own factory derives it at `VaultStackFactory.sol:869-871`.

The Lagoon fixture does not include `CuratorGuard`, so the ordinary deployed-capital posture is modelled by transferring 300 USDC out of the Safe. This preserves NAV while reducing Safe idle, which is the balance relationship the settlement and keeper coverage checks consume.

Relevant test:

```
// Alice makes the only legitimate request. This is what the keeper indexes.
vm.prank(alice);
vault.requestRedeem(aliceShares, alice, alice);

// Bob injects his shares straight into the silo. Plain ERC-20 transfer: no request, no
// RedeemRequest event, nothing for the keeper to index.
vm.prank(bob);
IShareTransfer(address(vault)).transfer(silo, bobShares);

assertEq(vault.pendingRedeemRequest(0, bob), 0, "bob has NO indexed request");
assertGe(safeBefore, keeperEstimate, "safe covers the keeper estimate");
assertLt(safeBefore, owed, "safe does NOT cover Lagoon aggregate");
```

Observed output:

```
[PASS] test_L18_eventlessSiloSharesEnterTheSettlementAggregate() (gas: 799595)
Logs:
  aliceRequestedShares  : 500000000000000000000
  bobInjectedShares     : 500000000000000000000
  bobIndexedRequest     : 0
  siloBalanceAfterRequest : 500000000000000000000
  siloBalanceAfterInject : 1000000000000000000000
  navTotalAssets        : 1000000000
  deployedAwayFromSafe  : 300000000
  safeIdleAtSettlement  : 700000000
  keeperCoverageBasis   : 500000000
  settleRedeem          : SUCCEEDED, no revert
  alicePendingAfter     : 500000000000000000000
  assetsPaid            : 0
```

```
[PASS] test_L18_negativeControl_withoutInjectionTheBatchSettles() (gas: 823965)
Logs:
  aliceRequestedShares   : 500000000000000000000
  bobInjectedShares      : 0
  settleRedeem           : SUCCEEDED
  alicePendingAfter      : 0
  assetsPaid             : 500000000

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.41ms (2.09ms CPU time)
```

Bob holds no indexed request at any point, yet his 500e18 shares double the silo balance from 500e18 to 1000e18 and enter the settlement aggregate. Safe idle of 700 USDC comfortably covers the keeper's request-derived basis of 500 USDC but not Lagoon's 1000 USDC aggregate, so `settleRedeem` returns successfully having paid nothing and Alice remains pending. The negative control runs the identical flow without the injection and the batch settles, paying 500 USDC, so the stall is caused by the eventless shares rather than the freeze, the funding, or the request.

The injector sacrifices the shares they transfer, so this is griefing and availability rather than theft, which is what holds it at Low.

The Foundry test directly proves the eventless injection, the larger Lagoon aggregate, and the silent non-settlement. The fixture does not run the keeper. The downstream custody hold is established by the production keeper's explicit no-op classification and error handling:

```
if (isSettleNoopFailure(e)) return { kind: 'settle_noop' }

e.kind === 'transient'
  ? Effect.void
  : repo.vaults.setCustodyHold(vaultId)
```

Therefore the PoC and the cited keeper path prove separate consecutive legs: Lagoon returns success without settling, the transport classifies the missing settlement event as `settle_noop`, and `processWithdrawal` persists the custody hold for every non-transient settlement failure.

Complete test source

`contracts/test/poc/L18SiloShareInjection.t.sol`

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.26;

import {console} from "forge-std/console.sol";
import {L15L16UnderfundedRedeemBatchTest} from "./L15L16UnderfundedRedeemBatch.t.sol";

interface IShareTransfer {
  function transfer(address to, uint256 amount) external returns (bool);
```

```
    function balanceOf(address account) external view returns (uint256);
}

/// L-18: Lagoon freezes the pending silo's ENTIRE ERC-20 share balance at valuation time
///
///  ERC7540Lib.sol:166
///    uint256 pendingShares = IERC20(address(this)).balanceOf(_pendingSilo);
///  ERC7540Lib.sol:174
///    $.settles[$.redeemSettleId].pendingShares = pendingShares;
///
/// and settles that aggregate (`ERC7540Lib.sol:690`). The silo is an ordinary ERC-20 holder, so an
/// access-permitted holder can move shares into it with a plain `transfer`, creating no controller
/// request and emitting no `RedeemRequest`. The keeper sizes required liquidity from its indexed
/// request rows, so those shares are inside Lagoon's settlement aggregate and outside the keeper's
/// coverage preflight.
contract L18SiloShareInjectionTest is L15L16UnderfundedRedeemBatchTest {
    /// The pending silo is the vault's first CREATE child. Same derivation the repository's own
    /// factory uses at `VaultStackFactory.sol:869-871`.
    function _pendingSilo(address vault_) internal pure returns (address) {
        return address(uint160(uint256(keccak256(abi.encodePacked(hex"d6", hex"94", vault_, hex"01")))));
    }

    /// Both holders in and settled, shares claimed.
    function _bothHoldShares() internal {
        vm.startPrank(alice);
        usdc.approve(address(vault), DEPOSIT_EACH);
        vault.requestDeposit(DEPOSIT_EACH, alice, alice);
        vm.stopPrank();
        vm.startPrank(bob);
        usdc.approve(address(vault), DEPOSIT_EACH);
        vault.requestDeposit(DEPOSIT_EACH, bob, bob);
        vm.stopPrank();
        vm.prank(valuationManager);
        vault.updateNewTotalAssets(0);
        vm.prank(safe);
        vault.settleDeposit(0);
        vm.prank(alice);
        vault.deposit(DEPOSIT_EACH, alice);
        vm.prank(bob);
        vault.deposit(DEPOSIT_EACH, bob);
    }

    function test_L18_eventlessSiloSharesEnterTheSettlementAggregate() public {
        _bothHoldShares();
        address silo = _pendingSilo(address(vault));

        // Alice makes the only legitimate request. This is what the keeper indexes.
        uint256 aliceShares = vault.balanceOf(alice);
        vm.prank(alice);
```

```
vault.requestRedeem(aliceShares, alice, alice);
uint256 siloAfterRequest = IShareTransfer(address(vault)).balanceOf(silo);

// Bob injects his shares straight into the silo. Plain ERC-20 transfer: no request, no
// RedeemRequest event, nothing for the keeper to index.
uint256 bobShares = vault.balanceOf(bob);
vm.prank(bob);
IShareTransfer(address(vault)).transfer(silo, bobShares);
uint256 siloAfterInjection = IShareTransfer(address(vault)).balanceOf(silo);

assertEq(siloAfterRequest, aliceShares, "silo holds only the requested shares");
assertEq(siloAfterInjection, aliceShares + bobShares, "injection lands in the same silo");
assertEq(vault.pendingRedeemRequest(0, bob), 0, "bob has NO indexed request");

// Ordinary custody posture: part of NAV is deployed away from the Safe, so Safe idle is
// below total assets. Modelled here as a transfer out, which is what allocateAndBridge
// does in production. NAV is unchanged; only its location is.
uint256 nav = vault.totalAssets();
uint256 deployed = 300e6;
vm.prank(safe);
usdc.transfer(makeAddr("coreAccount"), deployed);

// Valuation freezes the batch. Lagoon takes the whole silo balance, not the request set.
uint256 owed = nav;
vm.prank(valuationManager);
vault.updateNewTotalAssets(owed);

// The keeper sizes coverage from the requests it indexed, which is Alice's alone. Safe
// idle covers that comfortably.
uint256 keeperEstimate = (nav * aliceShares) / vault.totalSupply();
vm.prank(safe);
usdc.approve(address(vault), type(uint256).max);

uint256 safeBefore = usdc.balanceOf(safe);
assertGe(safeBefore, keeperEstimate, "safe covers the keeper estimate");
assertLt(safeBefore, owed, "safe does NOT cover Lagoon aggregate");
vm.prank(safe);
vault.settleRedeem(owed);

// Nothing settles: the aggregate needs the full silo balance.
assertEq(vault.pendingRedeemRequest(0, alice), aliceShares, "alice still pending");
assertEq(usdc.balanceOf(safe), safeBefore, "safe paid nothing");

console.log("aliceRequestedShares  :", aliceShares);
console.log("bobInjectedShares     :", bobShares);
console.log("bobIndexedRequest     :", vault.pendingRedeemRequest(0, bob));
console.log("siloBalanceAfterRequest :", siloAfterRequest);
console.log("siloBalanceAfterInject  :", siloAfterInjection);
console.log("navTotalAssets        :", owed);
```

```
    console.log("deployedAwayFromSafe   :", deployed);
    console.log("safeIdleAtSettlement   :", safeBefore);
    console.log("keeperCoverageBasis    :", keeperEstimate);
    console.log("settleRedeem          : SUCCEEDED, no revert");
    console.log("alicePendingAfter     :", vault.pendingRedeemRequest(0, alice));
    console.log("assetsPaid            :", safeBefore - usdc.balanceOf(safe));
  }

  /// Negative control: without the injection the identical flow settles, so the stall is caused
  /// by the eventless shares and not by the freeze, the funding, or the request itself.
  function test_L18_negativeControl_withoutInjectionTheBatchSettles() public {
    _bothHoldShares();

    uint256 aliceShares = vault.balanceOf(alice);
    vm.prank(alice);
    vault.requestRedeem(aliceShares, alice, alice);

    uint256 owed = vault.totalAssets();
    vm.prank(valuationManager);
    vault.updateNewTotalAssets(owed);
    vm.prank(safe);
    usdc.approve(address(vault), type(uint256).max);

    uint256 safeBefore = usdc.balanceOf(safe);
    vm.prank(safe);
    vault.settleRedeem(owed);

    assertEq(vault.pendingRedeemRequest(0, alice), 0, "alice settled");
    assertGt(safeBefore - usdc.balanceOf(safe), 0, "safe paid the batch");

    console.log("aliceRequestedShares  :", aliceShares);
    console.log("bobInjectedShares     : 0");
    console.log("settleRedeem          : SUCCEEDED");
    console.log("alicePendingAfter     :", vault.pendingRedeemRequest(0, alice));
    console.log("assetsPaid            :", safeBefore - usdc.balanceOf(safe));
  }
}
```

Recommendation

Derive the settlement obligation from chain-authoritative silo and epoch state, or reject settlement whenever the silo balance cannot be reconciled exactly to indexed request rows. The keeper should treat unexplained silo shares as a fail-closed discrepancy and raise a targeted alert before broadcasting settlement.

Team Response

Acknowledged.

I-01

INFO ACKNOWLEDGED

Two Role Changes for the Same Address Can Block Each Other

Description

A pending vault-leader address is not reserved by the protocol-role checks. This means address `X` can be proposed as the next vault leader and also be queued as the valuation manager, security council, or sanctions list.

Both proposals are accepted. After the delay, whichever change is activated first gives `X` its new role. The second change then fails because the address is already being used by another role.

Location of Affected Code

File: `contracts/src/libs/CuratorGuardIdentity.sol`

The identity check includes the proposed settle trigger but omits the proposed vault leader:

```
function _isDirectProtocolIdentity(address candidate, bool isPinned, RoleState memory roles) private view returns (bool)
{
  return _isFixedProtocolIdentity(candidate, isPinned, roles)
    || candidate == roles.settleTrigger
    || candidate == roles.proposedSettleTrigger;
}
```

File: `contracts/src/VaultOwnerTimelock.sol`

The timelock checks role collisions when the proposal is created and again when it is activated:

```
if (
  candidate == _leader()
    || guard.isTradingAgent(candidate)
    || guard.isProtocolIdentity(candidate)
    || _isVaultRole(candidate)
) revert RoleCollision(candidate);
```

File: `contracts/src/CuratorGuard.sol`

The leader change also checks its candidate again during activation:

```
function activateVaultLeaderChange(address newLeader) external {
  // Timelock and proposal checks omitted
  _requireValidVaultLeader(newLeader);

  _clearVaultLeaderProposal();
```

```
    vaultLeader = newLeader;
}
```

Impact

One planned role change can unexpectedly fail and must be cancelled or proposed again. This can delay either the leader rotation or the protocol-role rotation.

Proof of Concept

1. The vault leader proposes `X` as the next leader.
2. The steward also proposes `X` as the valuation manager.
3. Wait for both delays to finish.
4. Activate the valuation-manager change. It succeeds.
5. Activate the leader change. It reverts because `X` is now a vault role.

If the leader change is activated first, the valuation-manager change reverts instead. The same behavior applies to the security-council role and, when `X` is a contract, the sanctions-list role.

Recommendation

When a leader proposal is pending, reject that address from new protocol-role proposals.

This check should be added to the timelock's protocol-role validation instead of the global `isProtocolIdentity()` function, so a leader proposal does not block its own activation.

Team Response

Acknowledged.

I-02

INFO ACKNOWLEDGED

Fee-Receiver Rotation Omits Collision and Lagoon Admission Checks

Description

Fee-receiver configuration and rotation do not fully validate the new address against vault-specific protocol identities. Rotation checks only that the address is nonzero, applies none of the role-collision checks used by other protocol-role updates, and does not re-establish the Lagoon admission condition enforced by the factory at deployment.

One important collision is the vault's predicted `pendingSilo`. The factory reserves this address as a protocol identity when wiring the guard, but does not reject it as the initial `feeReceiver`; the timelock likewise does not reject it during rotation. If `pendingSilo` is selected, Lagoon mints fee shares into the address reserved for pending user redemptions. Those fee shares can then be treated as redemption shares during settlement, interfering with redemption accounting and settlement.

For an ordinary unadmitted receiver, newly minted fee shares do not by themselves freeze settlement. Under the whitelist access policy, however, that receiver cannot redeem the shares until the whitelist manager admits it. Transfers are not whitelist-gated in whitelist mode, so the shares can be moved; redemption is what is blocked.

Because configuration and rotation are privileged, and rotation is timelocked, visible, vetoable, and recoverable, this is a governance configuration footgun rather than an unprivileged funds attack.

Location of Affected Code

File: `contracts/src/VaultOwnerTimelock.sol`

```
function proposeUpdateFeeReceiver(address newReceiver) external onlyLeader returns (bytes32) {
  _requireNonZero(newReceiver);
  return _queueLeader(ILagoonVaultOwner.updateFeeReceiver.selector, abi.encode(newReceiver));
}

/// @notice Execute a matured, non-stale fee-receiver proposal.
function activateUpdateFeeReceiver(address newReceiver, uint256 nonce) external {
  _requireNonZero(newReceiver);
  _consumeLeader(ILagoonVaultOwner.updateFeeReceiver.selector, abi.encode(newReceiver), nonce);
  ILagoonVaultOwner(vault).updateFeeReceiver(newReceiver);
}
```

File: `contracts/src/VaultStackFactory.sol`

```
address[] memory feeBeneficiary = new address[](1);
feeBeneficiary[0] = cfg.feeReceiver;
vault.addToWhitelist(feeBeneficiary);
```

```
protocolIdentities[13] = _delayProxyAdmin(vault);
protocolIdentities[14] = _pendingSilo(vault);
```

```
if (!vault.isAllowed(cfg.feeReceiver)) revert PostconditionFailed(6);
```

Impact

A fee receiver can be configured as the pending redemption silo, causing fee shares to be mixed with shares reserved for user redemptions and disrupting redemption accounting or settlement. Other protocol-identity collisions may leave crystallized fee shares at an address without a supported recovery path. An unadmitted receiver cannot redeem under the current admission policy until admitted. User principal is not directly redirected.

Recommendation

Apply fee-receiver-specific role-collision checks both during factory preflight and during timelock proposal and activation. Explicitly reject the vault, predicted `pendingSilo`, delay proxy admin, and other incompatible protocol identities. Also require the receiver to be admitted at activation, or admit it atomically as part of rotation. Add Solidity tests for `pendingSilo`, other role-colliding receivers, and unadmitted receivers.

Team Response

Acknowledged.

I-03

INFO FIXED

No Boot Admission Check Ties Operated-Vault Count to the Read Budget the Repository Itself Models

Description

The `read-budget.ts` defines `computeReadBudget`, which models both `/info` REST weight and HyperEVM RPC request pressure and computes an `evmRpcOverBudget` verdict. It has zero production callers. The only references are its own module and its tests. The one adaptive control that is wired, `deriveAdaptiveCadences()`, is called from `config.ts` but is opt-in, with `KEEPER_ADAPTIVE_CADENCE` defaulting to false, and it deliberately adjusts only alert and indexer cadences, never valuation or settlement.

`config.ts` accepts any operated-vault count. The process boots and fans every tick across the whole fleet at bounded concurrency, which caps instantaneous fan-out but not per-minute request volume.

The modelled thresholds are not invented. Current Hyperliquid documentation states 1,200 aggregated REST weight per minute per IP and 100 EVM JSON-RPC requests per minute on the public HyperEVM endpoint. An executed sweep of the repository's own model has the busy-settlement, both-indexers scenario crossing the public EVM RPC ceiling at two vaults:

VAULT COUNT	EVM RPC REQ/MIN	OVER EVM CEILING	[VINFO] WEIGHT/MIN	OVER [VINFO] CEILING
1	52.67	no	193.33	no
2	101.33	yes	386.67	no
6	296.00	yes	1,160.00	no

The repository's own `read-budget.test.ts` pins the N=15 red cases and notes that even a 300 req/min managed-provider model is exceeded.

The two-vault threshold is conditional, not universal. The same executed model produced:

TWO-VAULT SCENARIO	EVM RPC REQ/MIN	OVER 100/MIN
Busy settlement + both indexers	101.33	yes
Idle settlement + both indexers	69.33	no
Busy settlement without both indexers	89.33	no
Idle settlement without both indexers	57.33	no

This matters for the admission-check design: the current model does not parameterize the actual RPC provider, shared egress topology, enabled indexer seams, or observed settlement duty cycle. A boot gate should consume those configured facts rather than applying the busy/all-indexer result unconditionally.

The shared-egress premise is not hypothetical either. `config.ts` records a live `-32005` rate-limit incident in which a shared-IP retry storm starved every other keeper read on the same IP, valuation included.

Location of Affected Code

File: `services/keeper/src/domain/read-budget.ts` – the model that exists and is never called in production:

```
/**
 * Compute the keeper's per-minute read fan-out at the model's N and cadences, per axis, and flag
 * whether either axis exceeds its ceiling. PURE — the bench asserts the numbers directly.
 */
export function computeReadBudget(
  model: ReadBudgetModel,
  limits: ReadBudgetLimits = DEFAULT_READ_BUDGET_LIMITS,
): ReadBudgetResult {
```

File: `services/keeper/src/config.ts` – the only wired lever, opt-in and alert-only, with no admission check on vault count:

```
const adaptiveCadence = parseEnvFlag(
 env.KEEPER_ADAPTIVE_CADENCE,
 'KEEPER_ADAPTIVE_CADENCE',
 false,
)
```

Impact

The reachability of fleet-wide impact is deployment-conditional: source does not prove production uses the public endpoint, a single shared egress IP, the modelled request counts, or a managed provider with the same ceiling. The confirmed defect needs none of that – the repository models a budget, tests it, and never enforces it at boot.

If the premise holds, the consequence is fleet-wide availability degradation: delayed valuation, settlement, indexing, ADL detection, and bridge driving across otherwise independent vaults, because the budget is shared per IP or provider and the scheduler retries on cadence. Money writes fail closed on missing evidence, so the impact is delayed operations, not loss.

Proof of Concept

Executed the production model directly with fixed default cadences and assertions for busy/idle settlement and indexer inclusion. The scenario matrix above passed.

Source and deployment inspection additionally confirmed:

- `computeReadBudget()` has no production caller;
- `KEEPER_ADAPTIVE_CADENCE` defaults to `false`;
- committed VPS and Terraform wiring does not enable that flag;
- Terraform supports the official public HyperEVM endpoint as the primary RPC;
- no committed deployment evidence establishes the currently deployed provider, shared egress, enabled seam combination, or fleet size.

The focused `read-budget.test.ts` suite remains green. This verification strengthens the missing-admission-control claim while preventing the N=2 worst-case model from being read as an unconditional deployment fact.

Recommendation

Add a fail-loud boot admission check that combines the operated-vault count, the enabled real seams, position counts, configured provider ceilings, and cadences, and refuses to boot – or requires an explicit acknowledgement – when the modelled load exceeds the configured ceiling. Reuse `computeReadBudget`, which already exists for this purpose. Make the provider ceiling an explicit configuration input rather than a constant in the model, and either arm adaptive cadence by default for multi-vault deployments or require explicit sharding.

Team Response

Fixed.

I-04

INFO FIXED

`proposeRaiseRates` Binds a Steward-Callable Operation to the Leader Epoch

Description

The `VaultOwnerTimelock` has two queueing helpers. `_queue()` assigns a nonce and an ETA. `_queueLeader()` calls `_queue()` and additionally stamps `opLeaderEpoch[opId]` with the current leader epoch; operations queued that way are consumed through `_consumeLeader()`, which reverts `StaleLeaderEpoch` if the epoch moved during the timelock window.

Every steward-owned family in this contract uses the epoch-free pair. `proposeUpdateValuationManager()`, `proposeUpdateSecurityCouncil()`, and `proposeUpdateSanctionsList` are all `onlySteward` and all call `_queue()`, so a leader rotation does not disturb them.

The `proposeRaiseRates()` is the exception. It is `onlyStewardOrLeader`, but it queues through `_queueLeader()`. A rate change proposed by the steward therefore carries the leader's epoch, and `activateRaiseRates()` reverts `StaleLeaderEpoch` if the leader rotated before activation. The current leader can rotate deliberately – by proposing a successor they control, letting the guard timelock mature, and letting anyone activate it – and can repeat that to force the steward to restart the full `TIMELOCK_DELAY` each time.

Location of Affected Code

File: `contracts/src/VaultOwnerTimelock.sol` – the steward-callable operation that carries the leader-epoch binding:

```
function proposeRaiseRates(Rates calldata newRates) external onlyStewardOrLeader returns (bytes32) {
  return _queueLeader(ILagoonVaultOwner.updateRates.selector, abi.encode(newRates));
}
```

The sibling steward-owned family that does not:

```
function proposeUpdateValuationManager(address newManager) external onlySteward returns (bytes32) {
  _requireValidProtocolRole(newManager);
  return _queue(ILagoonVaultOwner.updateValuationManager.selector, abi.encode(newManager));
}
```

Impact

This is held at Informational rather than Low for two reasons that both cut against the attacker.

Only fee **raises** are affected. `lowerRates()` is immediate, `onlyStewardOrLeader`, and carries no epoch binding, so risk-reducing changes can never be blocked by this mechanism. The griefing outcome is therefore depositor-favorable: it delays the protocol's own fee revenue, not user funds. The guardian can also veto the leader-rotation operations being used to drive the epoch forward.

What remains is a real family asymmetry. A steward-owned governance verb is silently subordinated to leader continuity in a contract where every other steward verb is not, and nothing in the NatSpec says so. An operator reading the contract would reasonably expect the steward's queued rate change to survive a leader rotation, as its three siblings do.

Recommendation

Either queue `proposeRaiseRates()` through `_queue()` so it matches the other steward-callable operations, or split the family into a leader-proposed lane that keeps the epoch binding and a steward-proposed lane that does not.

If the current coupling is deliberate, state in the NatSpec that a steward-proposed rate raise does not survive a leader rotation, and add a test that pins the behaviour so it is a decision rather than an accident.

Team Response

Fixed.

I-05

INFO FIXED

Agent-Rotation Activation Silently Supersedes a Pending Agent and Names Only the New One

Description

The `activateAgentChange()` assigns `pendingAgent` unconditionally. It never checks whether `pendingAgent` already held a different, unconfirmed agent, so an activation can replace an in-flight delegation the keeper has not yet confirmed against HyperCore. The emitted event carries only the incoming agent.

The vault-leader family in the same contract does the opposite. `activateVaultLeaderChange()` captures the outgoing identity before overwriting it, so its event can name both sides of the transition. The agent family cannot.

The stronger hypothesis for this shape – that supersession could produce a false confirmation – is rejected. The live keeper positively checks the HyperCore named slot before confirming, so a superseded agent is not confirmed as active. What remains is an observability gap on the one rotation family that can supersede.

Location of Affected Code

File: `contracts/src/CuratorGuard.sol` – `activateAgentChange`, which overwrites `pendingAgent` and emits only the destination:

```
_clearAgentProposal();
pendingAgent = agent;
_delegateAgent(agent);
emit AgentChangeActivated(agent);
```

`activateVaultLeaderChange()`, the sibling function that captures the predecessor:

```
address oldLeader = vaultLeader;
_clearVaultLeaderProposal();
vaultLeader = newLeader;
```

Impact

Reduced event provenance during incident response, on a custody-adjacent rotation. An operator reconstructing a rotation sequence from logs cannot tell, from the events alone, which delegation a given activation displaced. Where the leader family gives a complete transition record, the agent family gives a destination with no origin.

No dual authority is established and no false keeper confirmation follows, which is why this is Informational rather than a severity that implies a custody consequence.

Recommendation

Emit both the superseded and the incoming pending agent from `activateAgentChange()`, matching the vault-leader family's captured-predecessor pattern, and state in the NatSpec that supersession is intentional.

Alternatively, reject activation while another agent remains pending and unconfirmed, which would make the agent family match its two siblings rather than documenting why it differs.

Team Response

Fixed.

I-06

INFO FIXED

The Redemption Confirmation Path Overwrites the Recorded Transaction Hash on Replay

Description

The deposit and redemption confirmation paths disagree on whether a transaction hash survives a replayed observation. The deposit path preserves the first hash. The redemption path overwrites it with whatever the current event carries.

The redemption path is also internally inconsistent. The neighbouring field, `requestedAt`, is deliberately preserved on replay, with a comment explaining that resetting it would let a genuinely stuck redeem dodge the in-flight timeout. Two lines later a comment states that the `RedeemRequested` transaction is fixed at first observation and never superseded, while the code assigns `event.txHash` unconditionally. The comment describes the deposit path's behaviour, not this one's; the deposit path is where the preserve-on-replay rule is actually implemented.

Location of Affected Code

File: `services/keeper/src/application/confirm-pending-withdrawal.ts` – which overwrites and contradicts its own comment:

```
requestedAt: existing?.requestedAt ?? new Date(yield* clock.now),
status: 'PENDING',
// the RedeemRequested tx is fixed at first observation, never superseded;
// settledAt stays null until PASS 6 stamps the on-chain settle.
txHash: event.txHash,
```

File: `services/keeper/src/application/confirm-pending-deposit.ts` – which implements the rule that comment describes:

```
txHash: existing?.txHash ?? event.txHash,
```

Impact

Forensic-correlation inconsistency only. On-chain authority, settlement status, and the in-flight timeout are all unaffected: `requestedAt`, which the timeout actually reads, is preserved correctly on both paths.

The cost is that a replayed redemption observation rewrites the provenance an operator would use to correlate a durable row back to its originating transaction, and that a reader of this file is told the opposite of what the code does. The comment is the more durable problem of the two, because it will be trusted by the next person to touch this path.

Recommendation

Apply the deposit path's preserve-on-replay rule to the redemption path so the code matches its own comment, and add a test asserting that a repeated observation against an existing row does not rewrite the stored hash.

If overwriting is in fact intended for redemptions, correct the comment instead, and say why the two paths differ.

Team Response

Fixed.

I-07

INFO FIXED

Redemptions Can Remain Pending Because Silent No-Ops Emit Generic Success

Description

Lagoon's redemption settlement path can return without settling when the Safe lacks sufficient liquidity. The call does not revert in that case. `RiniSettleModule` then emits its generic `Settled` event because the downstream call itself completed successfully.

The wrapper event therefore does not prove that a redemption batch actually progressed. Current keeper logic mitigates the problem by prechecking liquidity and requiring Lagoon lifecycle evidence, but integrations or future code that rely on the module event alone can interpret a no-op as success.

Location of Affected Code

File: `contracts/src/RiniSettleModule.sol:123-155`

File: `contracts/lib/lagoon-v0/src/v0.6.0/libraries/ERC7540Lib.sol:680-735`

File: `services/keeper/src/infrastructure/adapters/settle-transports.adapter.ts:291-307`

File: `services/keeper/src/application/process-withdrawal.ts:308-416`

Impact

A consumer can report successful settlement while withdrawals remain pending, causing delayed detection and unnecessary retries. Existing keeper checks reduce current exploitability, and no funds are directly lost.

The same silent no-op is the terminal mechanism behind the custody holds described in M-01 and L-18.

Recommendation

Have the module verify an operation-specific Lagoon event or post-state change before emitting success. If a batch does not progress, emit an explicit no-op result or revert with a distinct error.

Team Response

Fixed.

I-08

INFO FIXED

Role Changes Can Be Delayed Because the Timelock Retains a Cleared Sanctions Oracle

Description

The whitelist manager can clear the vault's live sanctions-oracle pointer directly. The owner timelock retains a cached oracle identity that is still considered part of the protocol's role set.

After the live pointer is cleared, that cached identity can prevent the same address from being proposed immediately for another role, even though it no longer exercises oracle authority in the vault. Restoring the oracle or selecting a replacement can require an additional delayed governance cycle.

The steward can set and later restore the live pointer, so the condition is recoverable. The inconsistency nevertheless causes avoidable role lockout and makes identity validation depend on stale mirrored state.

Location of Affected Code

File: `contracts/src/VaultOwnerTimelock.sol:319-342,505-516`

File: `contracts/src/libs/CuratorGuardWiring.sol:43-60`

Impact

Routine oracle maintenance can make an address unavailable for other roles and add an unnecessary timelock cycle to recovery. No unauthorized action or direct loss results.

Recommendation

Validate oracle identity against the vault's live pointer, or synchronize the cached value whenever the live configuration changes. If the previous oracle must remain reserved for safety, represent that state explicitly with a bounded lifecycle rather than an implicit stale mirror.

Team Response

Fixed.

I-09

INFO FIXED

V4 Timelock Operations Can Be Missed Because the Watcher Uses a Stale Event ABI

Description

The generic `CuratorGuard` watcher filters a two-argument `PrivilegedOpProposed` event. The V4 contract emits the same event name with three arguments, producing a different topic signature.

Because the watcher installs filters for the stale signature, no V4 privileged-operation logs match. The source returns an empty successful result and the watcher reports zero discovered, open, or matured operations instead of identifying an ABI mismatch.

The conservation watcher is not a full substitute. It ignores outbound reconciliations and return reconciliations that do not match its narrow double-release pattern. Guardian approval remains required for execution, but operator visibility into the complete V4 privileged-operation queue is lost.

Location of Affected Code

File: `contracts/src/CuratorGuard.sol:178-192,602-755`

File: `packages/abis/src/curator-guard-timelock-events.abi.ts:17-19,73-79`

File: `services/keeper/src/infrastructure/adapters/guard-timelock-event-source.adapter.ts:57-70,180-192`

File: `services/keeper/src/application/watch-guard-timelock.ts:496-624`

The deployed Guard emits a three-argument event:

```
// contracts/src/CuratorGuard.sol:192
event PrivilegedOpProposed(bytes32 indexed opId, uint256 eta, uint256 expiresAt);
```

The keeper derives its filter `topic0` from a two-argument fragment in `packages/abis/src/curator-guard-timelock-events.abi.ts:74-78`, consumed at `guard-timelock-event-source.adapter.ts:62-64`.

Impact

The watcher can return a false-clean state while V4 reconcile operations are live or mature. On-chain guardian approval remains a backstop, limiting the effect to monitoring and decision quality rather than unilateral execution.

Proof of Concept

The runnable PoC is located at:

```
services/keeper/src/poc/I09-privileged-op-topic0-mismatch.poc.test.ts
```

Run:

```
cd services/keeper && pnpm vitest run src/poc/I09-privileged-op-topic0-mismatch.poc.test.ts
```

The deployed Guard emits a THREE-argument event (`CuratorGuard.sol:192`):

```
event PrivilegedOpProposed(bytes32 indexed opId, uint256 eta, uint256 expiresAt);
```

The keeper derives its filter topic0 from a TWO-argument fragment (`packages/abis/src/curator-guard-timelock-events.abi.ts:74-78`), consumed at `guard-timelock-event-source.adapter.ts:62-64`.

Observed output:

```
deployed event      : PrivilegedOpProposed(bytes32,uint256,uint256)
deployed topic0     : 0xf394f86b621a90071dcd5d497416aed8347a578bdd2b22ea2920729e91910112
keeper filter event : PrivilegedOpProposed(bytes32,uint256)
keeper filter topic0: 0x2b4077d3ac332a40e4a4d95d2125114284ba2ce681a7211e0f4a600b494faeb2
topics match        : false
consequence         : no deployed log can match the installed filter
corrected filter event : PrivilegedOpProposed(bytes32,uint256,uint256)
corrected topic0    : 0xf394f86b621a90071dcd5d497416aed8347a578bdd2b22ea2920729e91910112
deployed topic0     : 0xf394f86b621a90071dcd5d497416aed8347a578bdd2b22ea2920729e91910112
topics match        : true
twoArgTopic0        : 0x2b4077d3ac332a40e4a4d95d2125114284ba2ce681a7211e0f4a600b494faeb2
threeArgTopic0      : 0xf394f86b621a90071dcd5d497416aed8347a578bdd2b22ea2920729e91910112
sharedPrefixBytes   : none
silentFailureMode   : filter installs cleanly and matches nothing
```

The two topic0 values differ, so no deployed log can match the installed filter. The first control shows the corrected three-argument signature reproduces the deployed topic0 exactly, which isolates the missing third argument as the cause rather than anything about the derivation. The failure is silent: the filter installs cleanly and matches nothing, so the watcher reports zero discovered, open or matured operations.

Complete test source

services/keeper/src/poc/l09-privileged-op-topic0-mismatch.poc.test.ts

```
import { toEventSelector } from 'viem'
import { describe, expect, it } from 'vitest'

// L-09: the deployed V4 Guard emits a THREE-argument event
//  CuratorGuard.sol:192
//    event PrivilegedOpProposed(bytes32 indexed opId, uint256 eta, uint256 expiresAt);
//
// while the keeper derives its filter topic0 from a TWO-argument fragment
//  packages/abis/src/curator-guard-timelock-events.abi.ts:74-78
//    PrivilegedOpProposed(bytes32 indexed opId, uint256 eta)
//  consumed at guard-timelock-event-source.adapter.ts:62-64 via toEventSelector.
//
// topic0 is keccak256 over the canonical signature INCLUDING every argument type, so the two
// differ and no deployed log can match the installed filter. The adapter returns a successful
// empty result rather than an ABI mismatch, so the watcher reports zero discovered, open or
// matured operations.

const DEPLOYED = 'PrivilegedOpProposed(bytes32,uint256,uint256)'
const KEEPER_FILTER = 'PrivilegedOpProposed(bytes32,uint256)'

describe('L-09 privileged-op watcher filters a stale signature', () => {
 it('the deployed event topic0 and the keeper filter topic0 differ', () => {
  const deployedTopic0 = toEventSelector(DEPLOYED)
  const keeperTopic0 = toEventSelector(KEEPER_FILTER)

  console.log('deployed event       :', DEPLOYED)
  console.log('deployed topic0      :', deployedTopic0)
  console.log('keeper filter event  :', KEEPER_FILTER)
  console.log('keeper filter topic0 :', keeperTopic0)
  console.log('topics match         :', deployedTopic0 === keeperTopic0)
  console.log('consequence          : no deployed log can match the installed filter')

  expect(deployedTopic0).not.toBe(keeperTopic0)
 })

 it('negative control: the matching signature reproduces the deployed topic0 exactly', () => {
  // Same derivation, correct argument list. This shows the mismatch is the third argument and
  // not something about how the selector is computed.
  const corrected = toEventSelector('PrivilegedOpProposed(bytes32,uint256,uint256)')
```

```
  const deployedTopic0 = toEventSelector(DEPLOYED)

  console.log('corrected filter event  : PrivilegedOpProposed(bytes32,uint256,uint256)')
  console.log('corrected topic0      :', corrected)
  console.log('deployed topic0       :', deployedTopic0)
  console.log('topics match          :', corrected === deployedTopic0)

  expect(corrected).toBe(deployedTopic0)
 })

 it('second control: a one-argument difference is sufficient to break the match', () => {
  // Demonstrates the general property rather than asserting it: adding any argument changes
  // topic0, which is why a signature drift is silent rather than loud.
  const twoArg = toEventSelector('PrivilegedOpProposed(bytes32,uint256)')
  const threeArg = toEventSelector('PrivilegedOpProposed(bytes32,uint256,uint256)')

  console.log('twoArgTopic0          :', twoArg)
  console.log('threeArgTopic0        :', threeArg)
  console.log('sharedPrefixBytes     :', twoArg.slice(0, 4) === threeArg.slice(0, 4) ? 'none beyond 0x' : 'none')
  console.log('silentFailureMode     : filter installs cleanly and matches nothing')

  expect(twoArg).not.toBe(threeArg)
 })
})
```

Recommendation

Replace the stale event fragment with the exact canonical V4 ABI and fail boot when the configured event signature does not match the deployed generation. Add a fixture that emits the three-argument V4 event and assert that discovery, maturity, and cleanup are all observed.

Team Response

Fixed.



Your security partner, for the long term

SHIELDIFY.ORG · X.COM/SHIELDIFYSEC · LINKEDIN